



CAK2HAB3 - Dasar Kecerdasan Artifisial

SEARCHING

Heuristic Search

Prof. Suyanto

S1 Informatika – Fakultas Informatika

Semester Genap 2024/2025





Heuristic Search

- 
- *Heuristic* berasal dari bahasa Yunani, *heuriskein*, yang berarti ‘mencari’ atau ‘menemukan’
 - Pada teknik pencarian, *heuristic* diartikan sebagai biaya estimasi dari suatu titik ke solusi
 - Pada banyak kasus, *heuristic* mampu mempercepat proses pencarian secara signifikan

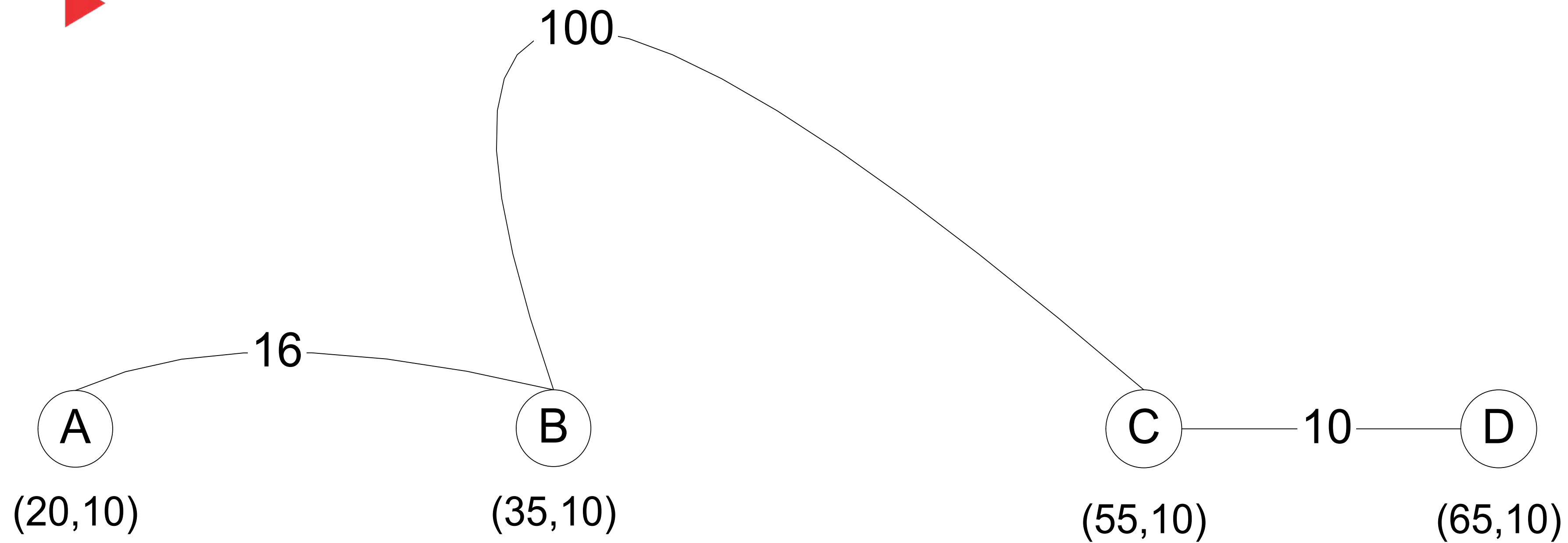




Heuristic Search

- Fungsi heuristik harus menghasilkan biaya estimasi yang lebih kecil atau sama dengan biaya sebenarnya
 - Jika tidak, maka proses pencarian bisa tersesat dan tidak optimal
 - Fungsi heuristik dikatakan baik jika memberikan biaya estimasi yang mendekati biaya sebenarnya
 - Semakin mendekati biaya sebenarnya, fungsi heuristik tersebut semakin baik
- 

Heuristic Search



Fungsi heuristik h berupa jarak garis lurus atau Euclidean distance:

$$d_{ab} = \sqrt{(y_b - y_a)^2 + (x_b - x_a)^2}$$



Heuristic Search

- 
- ▶ 1. Hill Climbing
 - 2. Simulated Annealing
 - 3. Greedy Best-First Search
 - 4. A^*



CAK2HAB3 - Dasar Kecerdasan Artifisial

Hill Climbing

S1 Informatika – Fakultas Informatika

Semester Genap 2024/2025

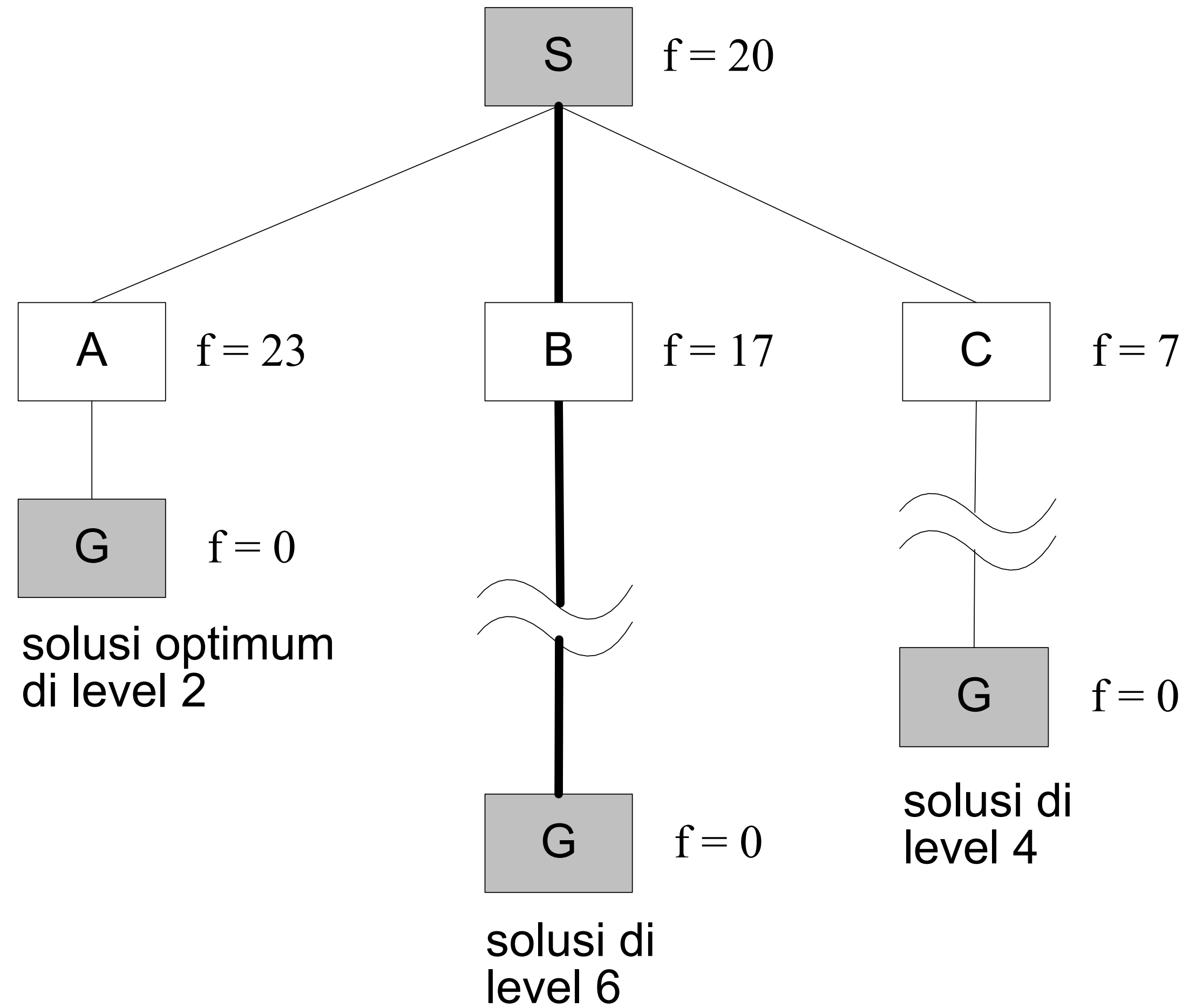




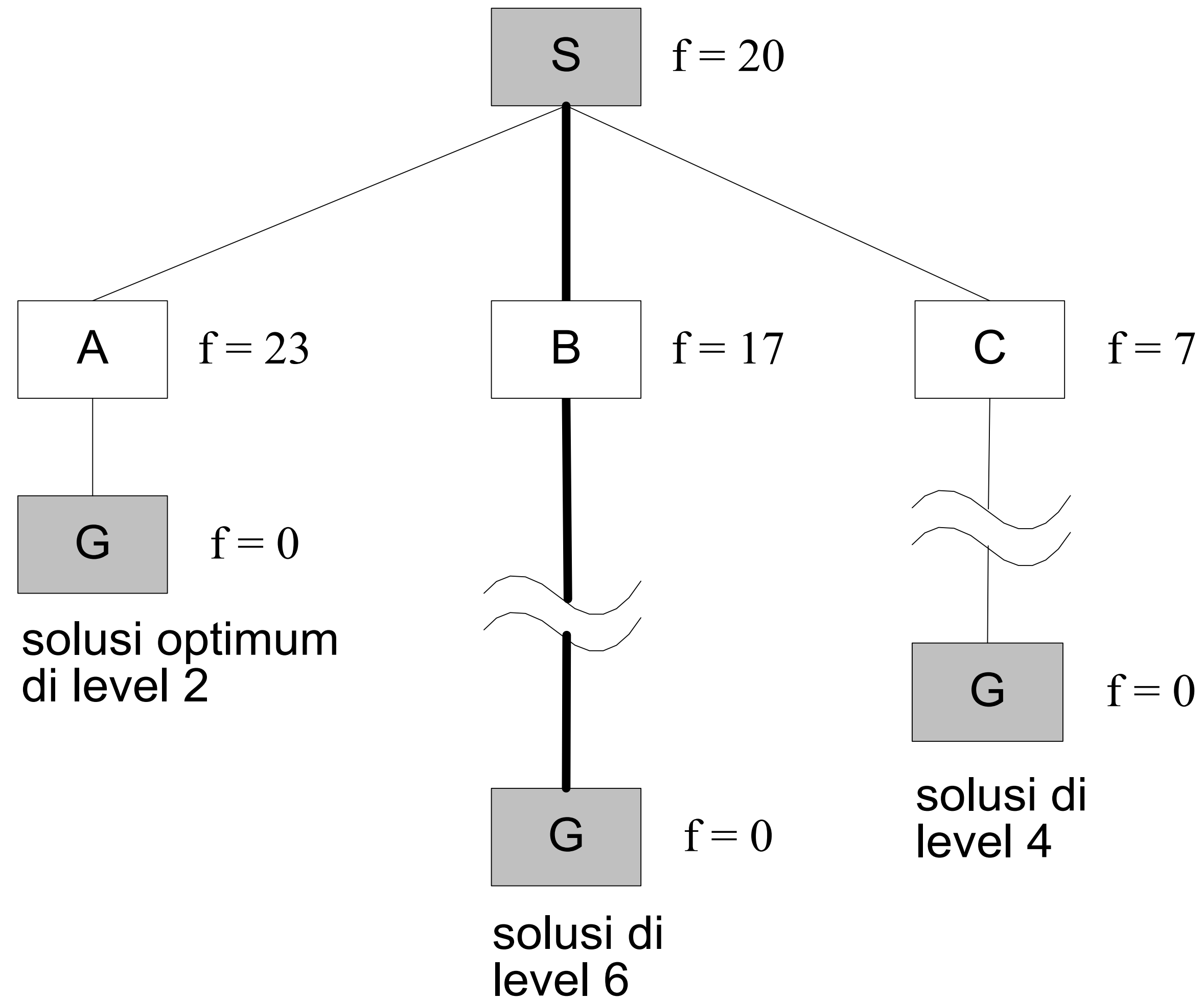
Hill Climbing

- ▶ **Memerlukan fungsi** heuristik untuk mengevaluasi state
 - Simple Hill Climbing
 - Steepest-Ascent Hill Climbing
- 

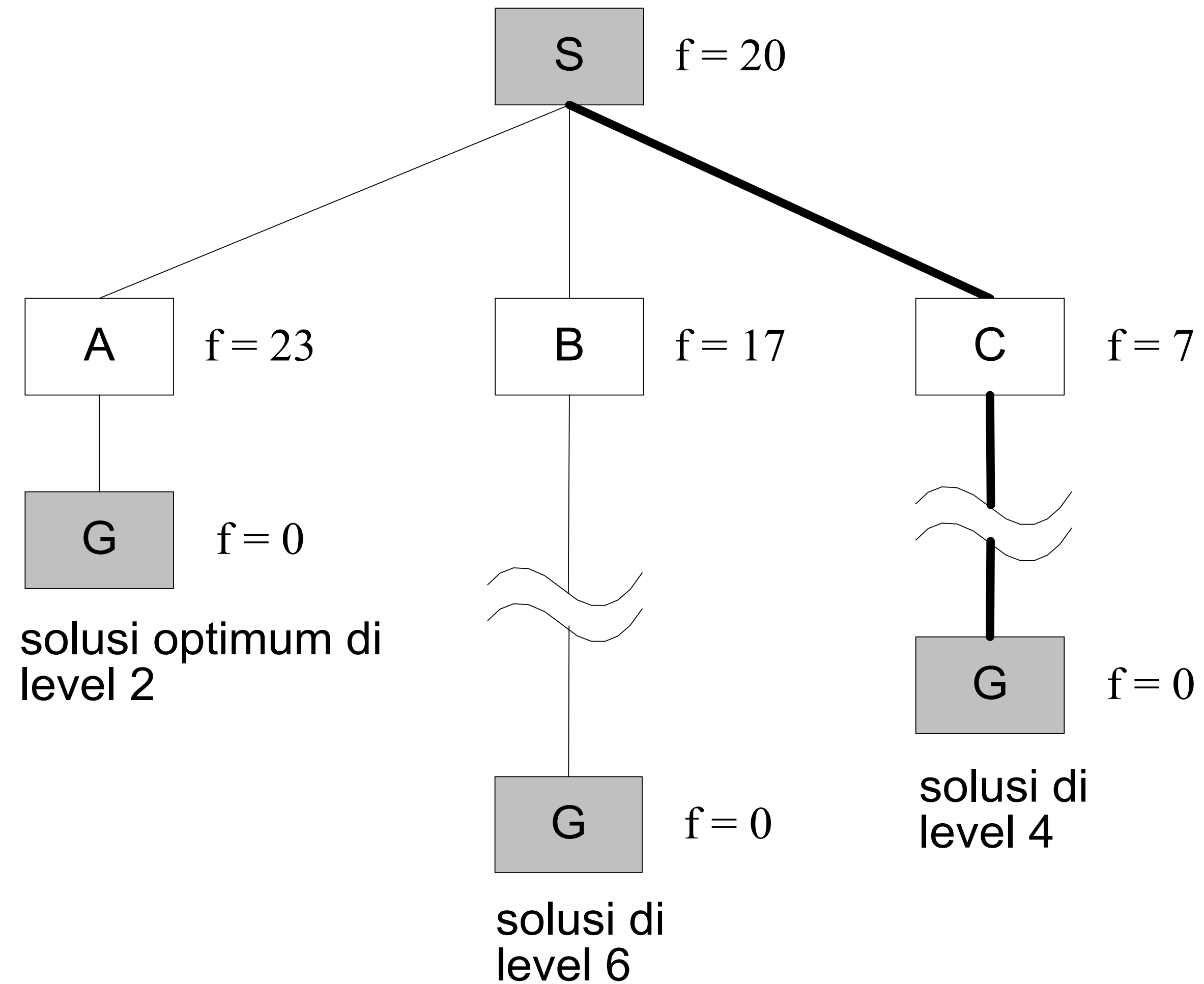
Simple Hill Climbing



Simple Hill Climbing



Steepest-Ascent Hill Climbing





CAK2HAB3 - Dasar Kecerdasan Artifisial

Simulated Annealing

S1 Informatika – Fakultas Informatika

Semester Genap 2024/2025





Simulated Annealing

- 
- Analogi cara pendinginan dan pembekuan metal atau kristal dengan energi yang minimal
 - Dengan probabilitas tertentu, Simulated Annealing bisa keluar dari jebakan minimum lokal
 - Simulated Annealing menggunakan sebuah rumus probabilitas yang memungkinkannya bisa keluar dari minimum lokal
 - *New state* yang tidak lebih baik dari *current state* masih mungkin dipilih dengan probabilitas

$$p(\Delta E) = e^{-\Delta E / T} \quad \text{di mana } \Delta E = f(\text{current state}) - f(\text{new state})$$

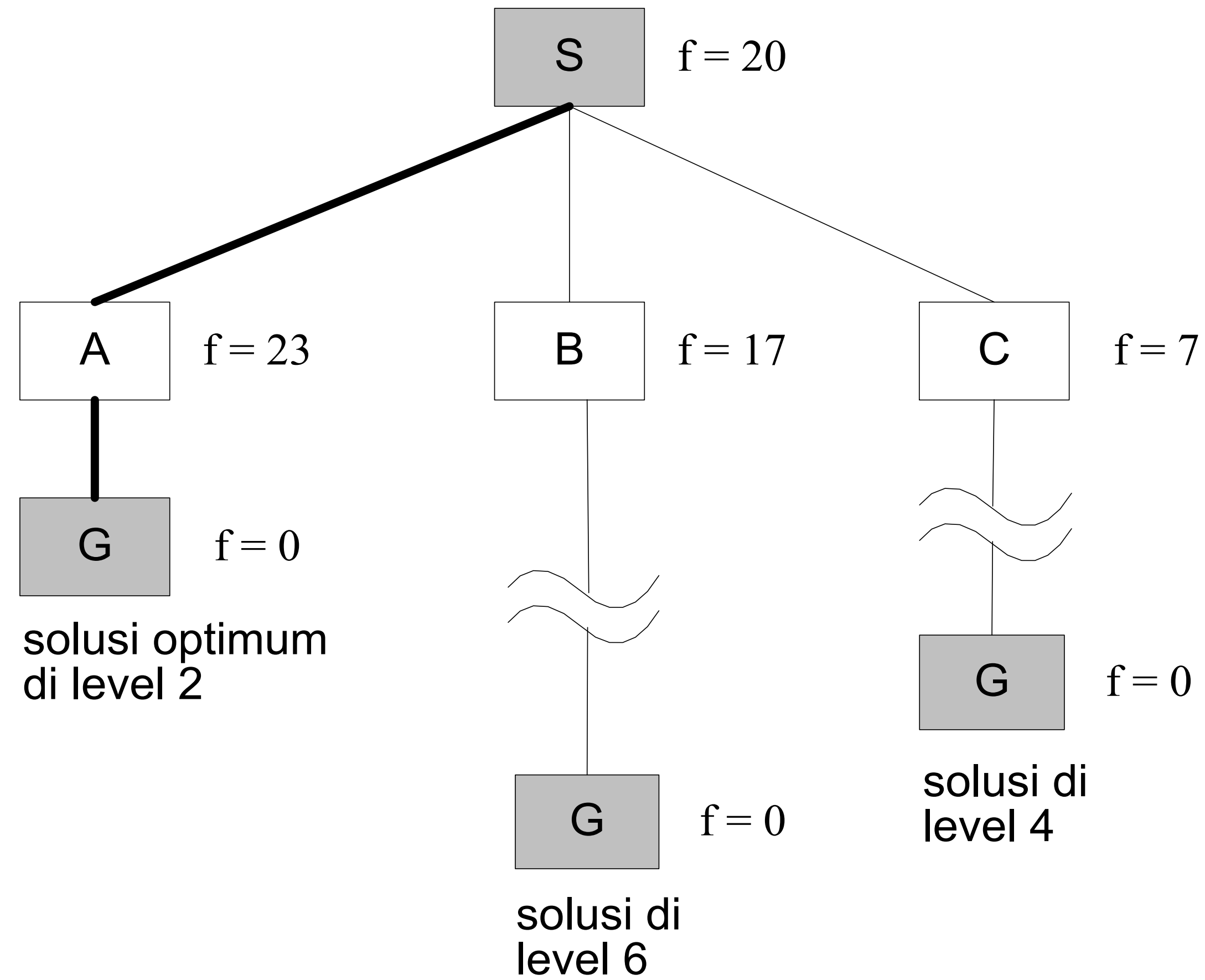



Simulated Annealing

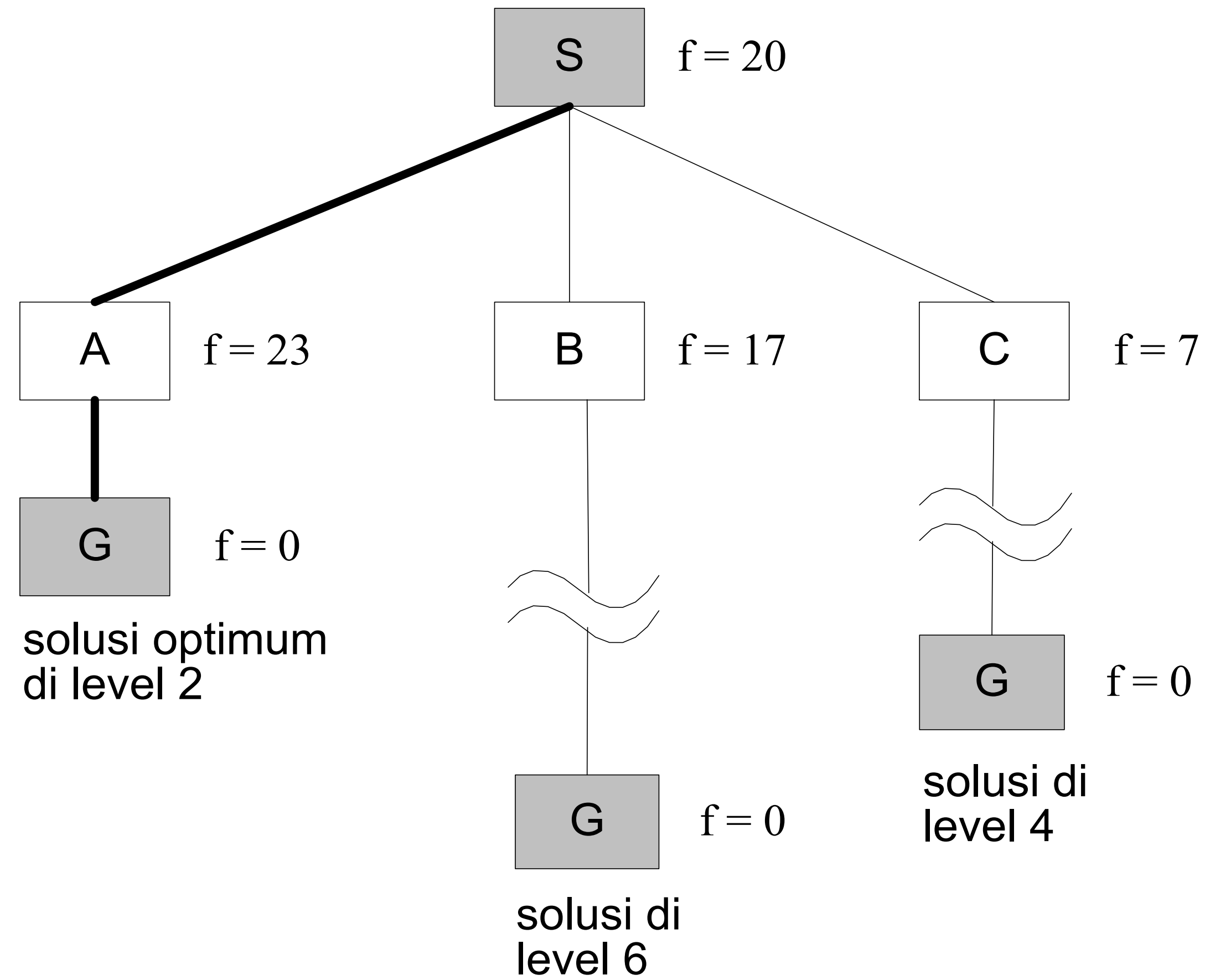
- 
- Analogi cara pendinginan dan pembekuan metal atau kristal dengan energi yang minimal
 - Dengan probabilitas tertentu, Simulated Annealing bisa keluar dari jebakan minimum lokal
 - Simulated Annealing menggunakan sebuah rumus probabilitas yang memungkinkannya bisa keluar dari minimum lokal
 - *New state* yang tidak lebih baik dari *current state* masih mungkin dipilih dengan probabilitas

$$p(\Delta E) = e^{-\Delta E / T} \quad \text{di mana } \Delta E = f(\text{current state}) - f(\text{new state})$$


Simulated Annealing



Simulated Annealing





CAK2HAB3 - Dasar Kecerdasan Artifisial

Greedy Best First Search

S1 Informatika – Fakultas Informatika

Semester Genap 2024/2025





Greedy Best First Search

- 
- Memilih simpul baru yang memiliki biaya terkecil diantara semua *leaf nodes*
 - Membangkitkan simpul berikutnya dari sebuah simpul, yang sejauh ini, terbaik diantara semua leaf nodes yang pernah dibangkitkan
 - Bagaimana menentukan sebuah simpul terbaik saat ini?
 - Penentuan simpul terbaik dilakukan hanya berdasarkan nilai heuristik atau **biaya estimasi** dari suatu simpul menuju ke goal. Tidak memperhitungkan **biaya sebenarnya**.
- 

$$f(n) = h(n)$$

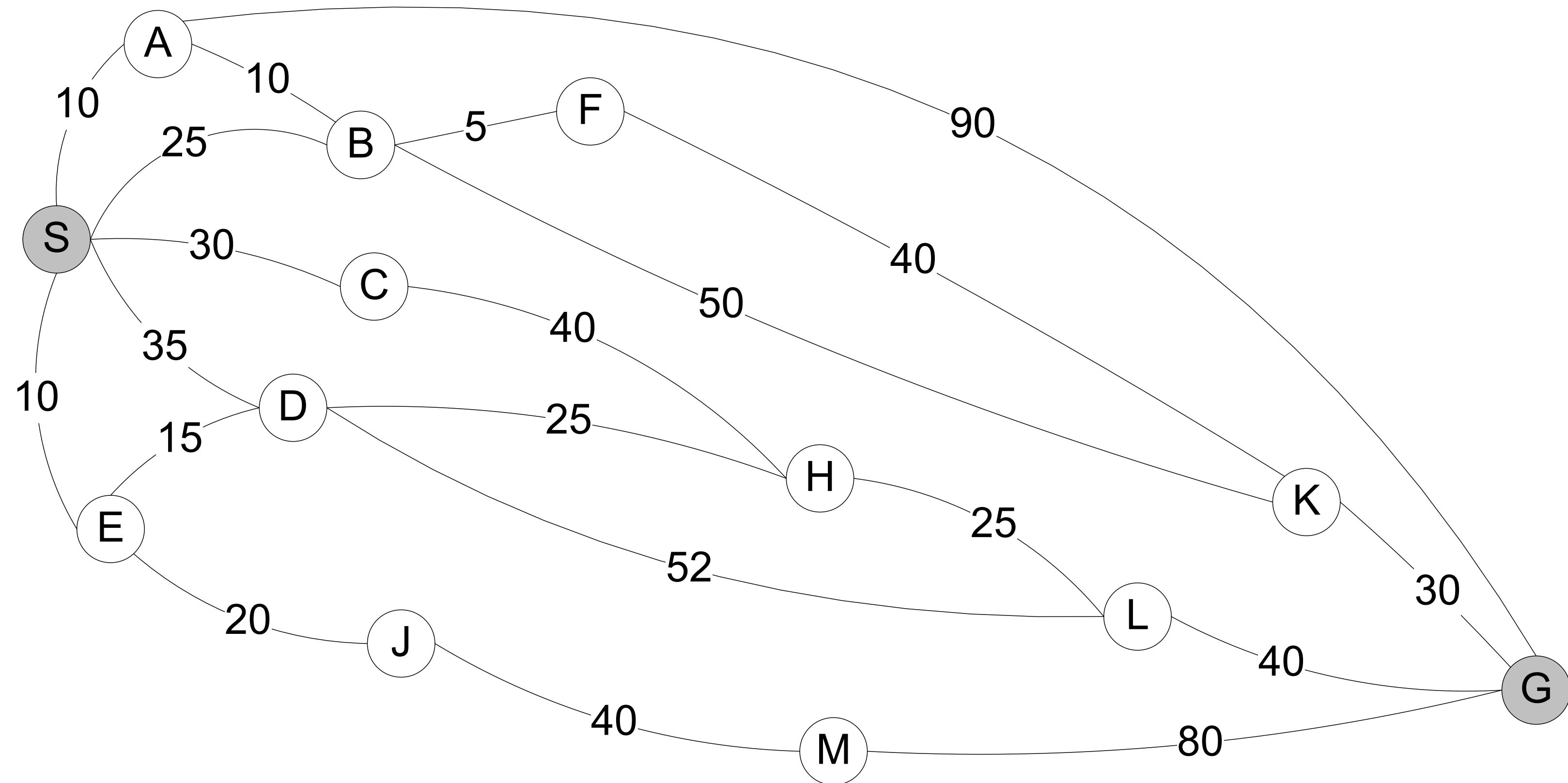


Greedy Best First Search

- ▶ Memilih simpul baru yang memiliki biaya terkecil diantara semua *leaf nodes*
- Membangkitkan simpul berikutnya dari sebuah simpul, yang sejauh ini, terbaik diantara semua leaf nodes yang pernah dibangkitkan
- Bagaimana menentukan sebuah simpul terbaik saat ini?
- Penentuan simpul terbaik dilakukan hanya berdasarkan nilai heuristik atau **biaya estimasi** dari suatu simpul menuju ke goal. Tidak memperhitungkan **biaya sebenarnya**.

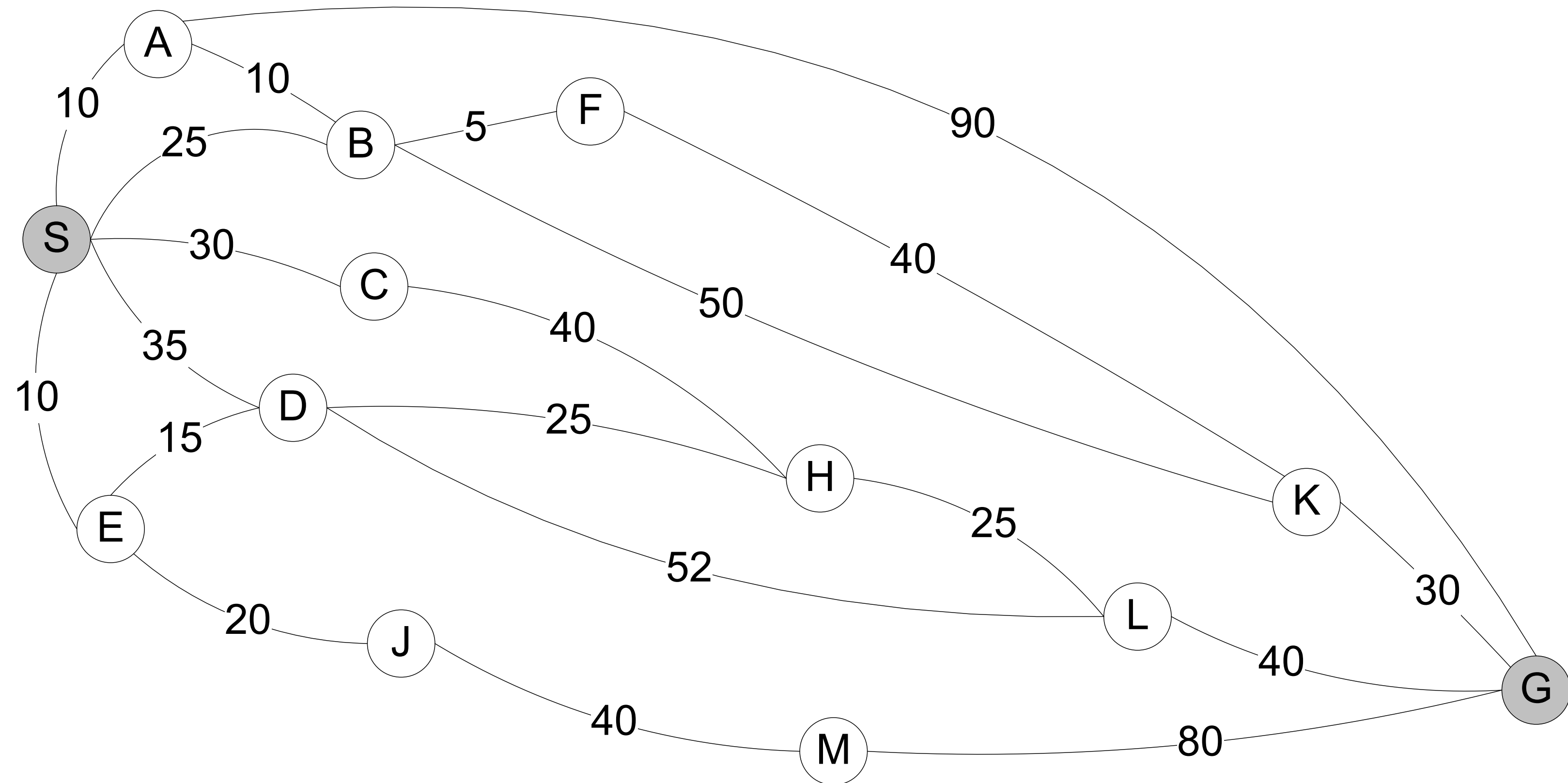
$$f(n) = h(n)$$


Greedy Best First Search



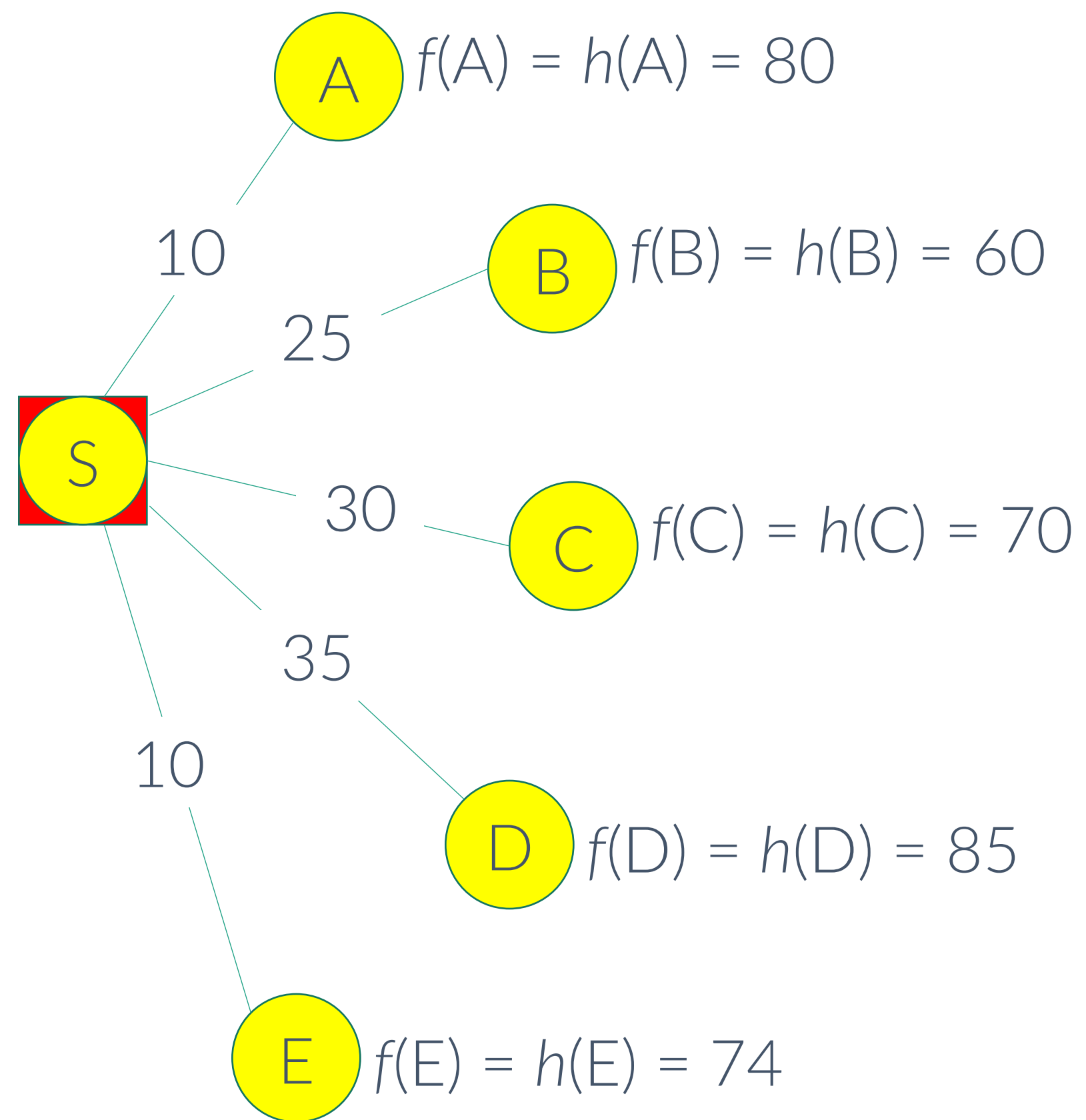
n	S	A	B	C	D	E	F	G	H	J	K	L	M
$h(n)$	80	80	60	70	85	74	70	0	40	100	30	20	70

Greedy Best First Search

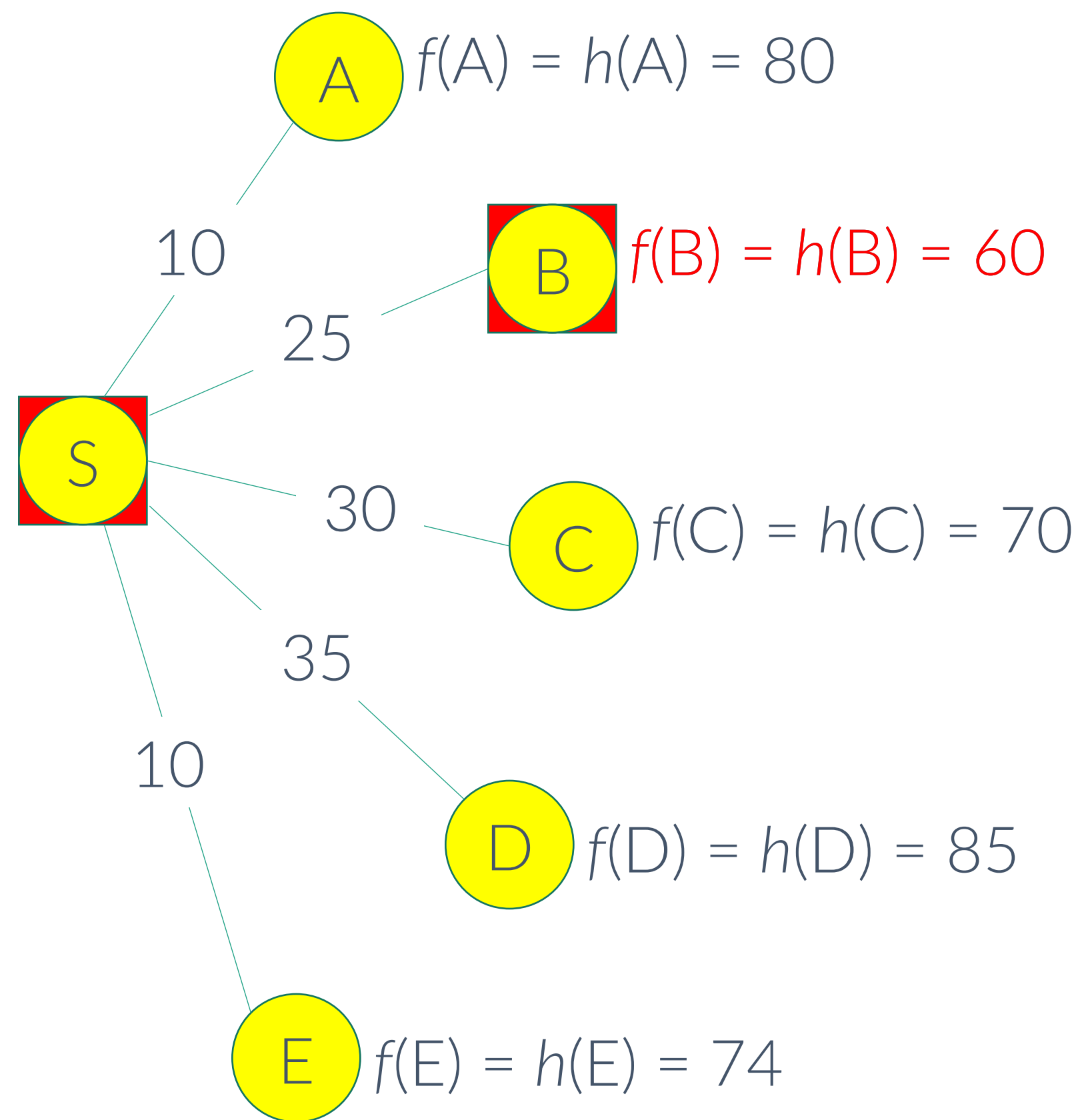


n	S	A	B	C	D	E	F	G	H	J	K	L	M
$h(n)$	80	80	60	70	85	74	70	0	40	100	30	20	70

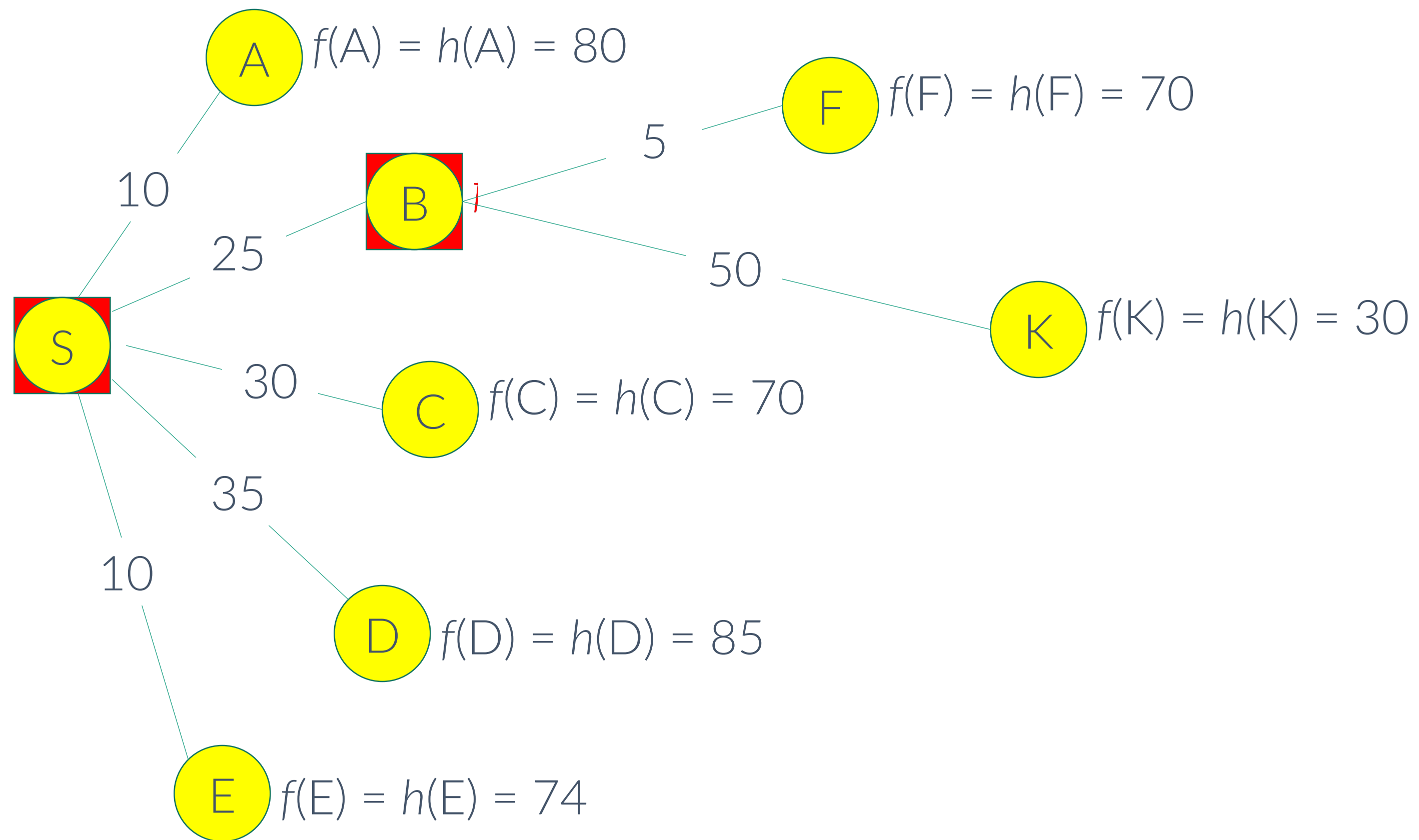
Greedy Best First Search



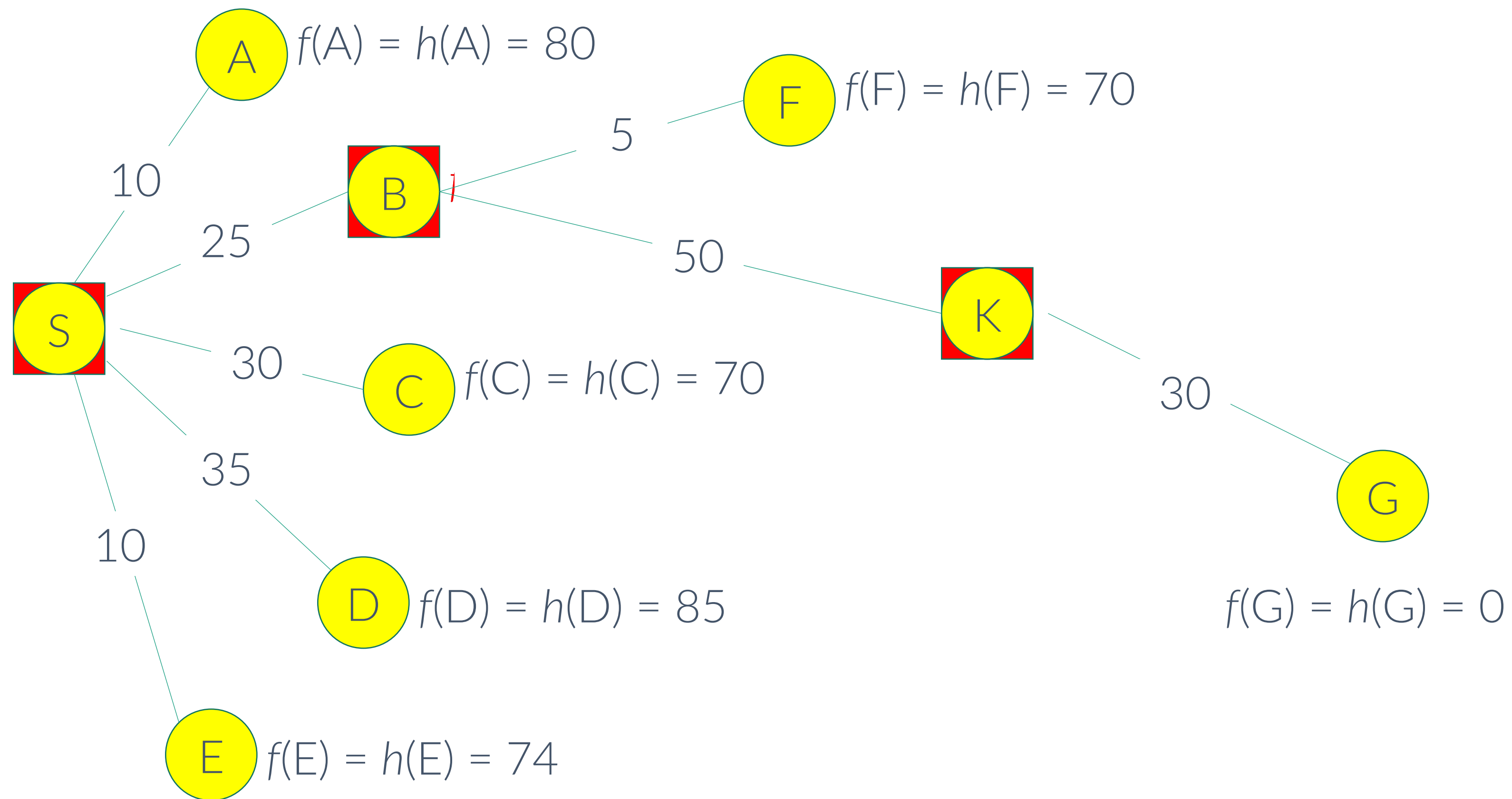
Greedy Best First Search



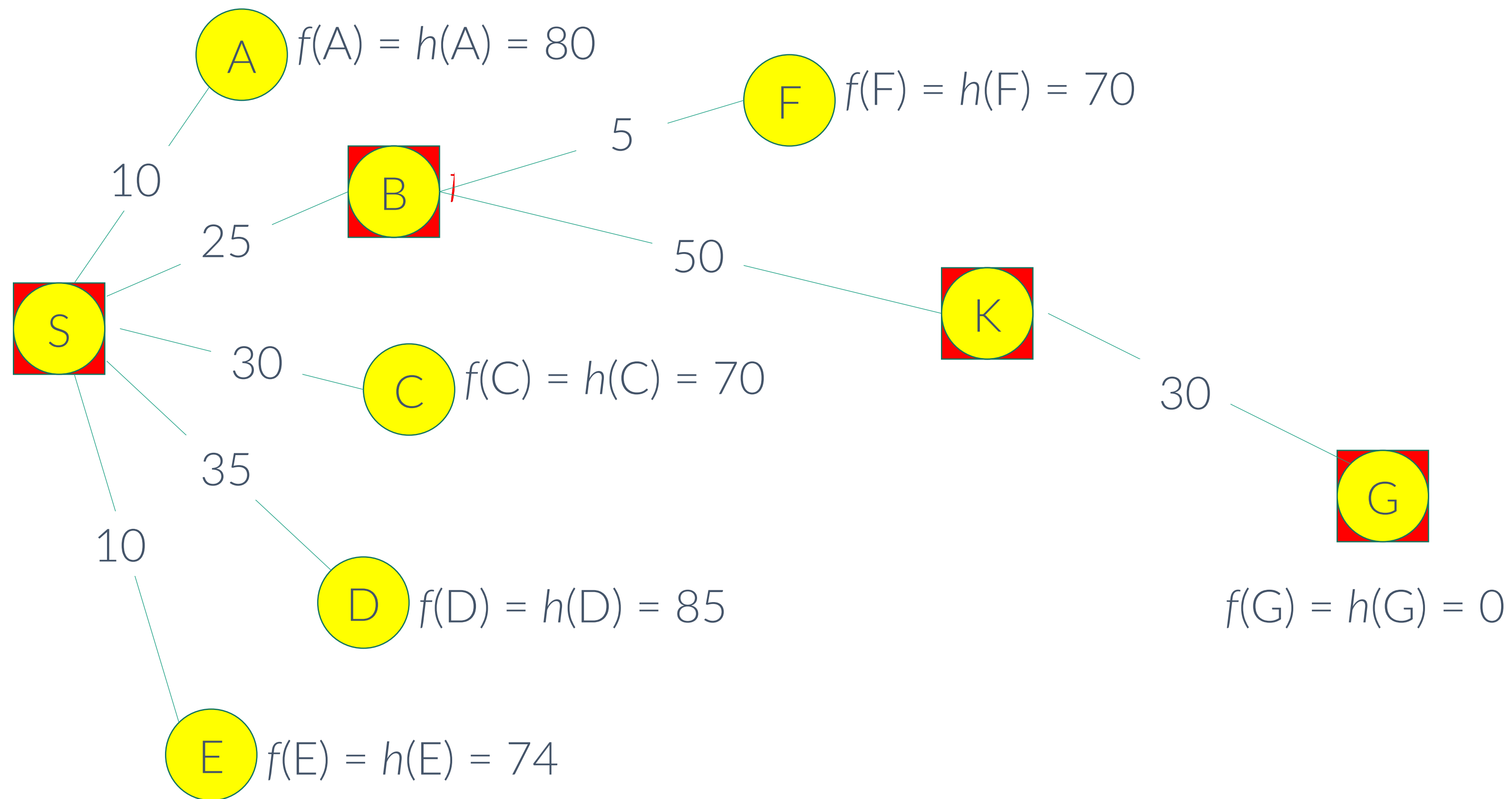
Greedy Best-First Search



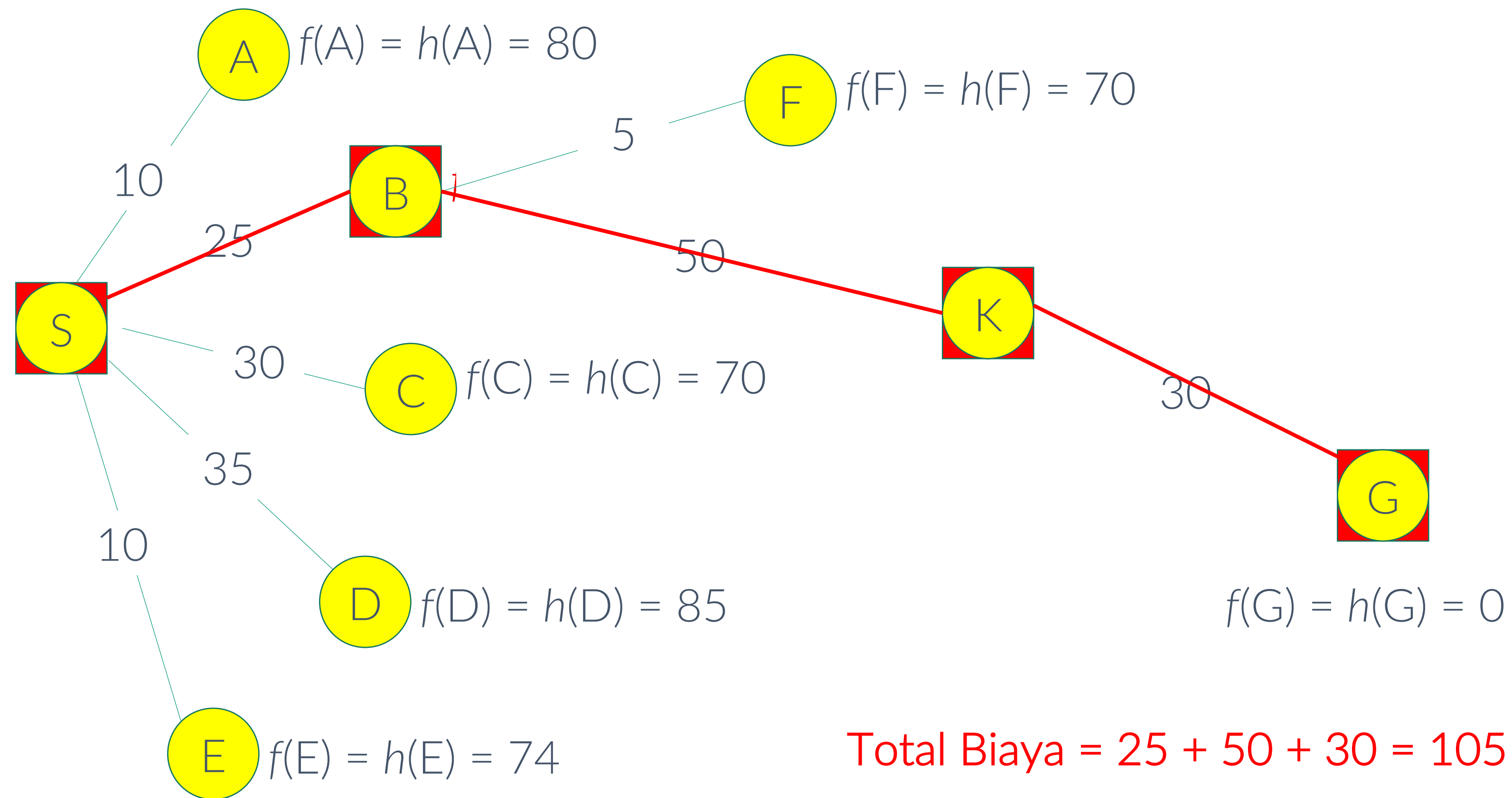
Greedy Best-First Search



Greedy Best-First Search



Greedy Best-First Search



Total Biaya = $25 + 50 + 30 = 105$

Biaya paling optimal = 95

CAK2HAB3 - Dasar Kecerdasan Artifisial

A^*

S1 Informatika – Fakultas Informatika

Semester Genap 2024/2025





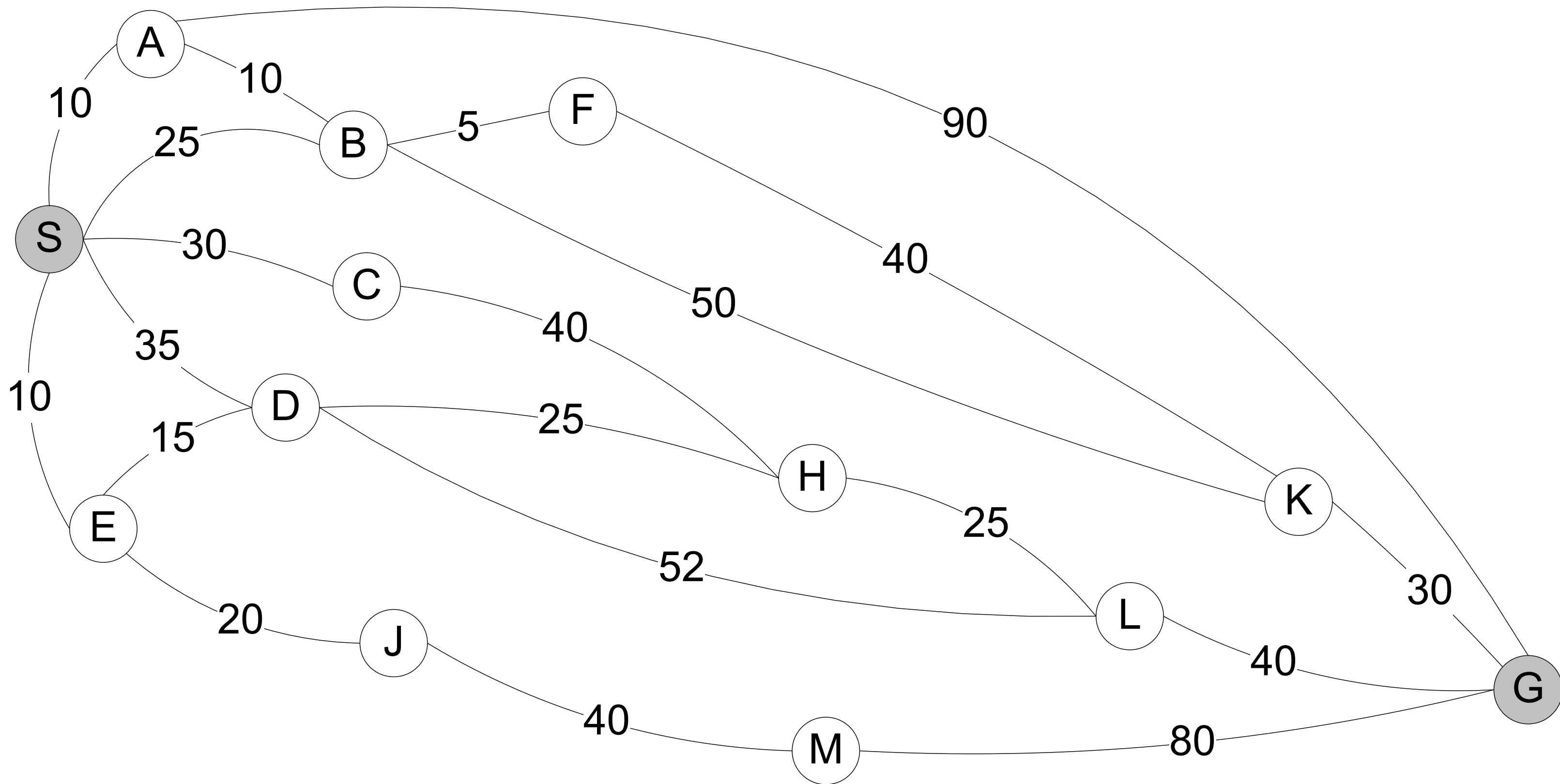
A^*

- 
- ▶ Gabungan *Uniform Cost Search* dan *Greedy Best-First Search*
 - Biaya = biaya sebenarnya ditambah biaya estimasi

$$f(n) = g(n) + h(n)$$

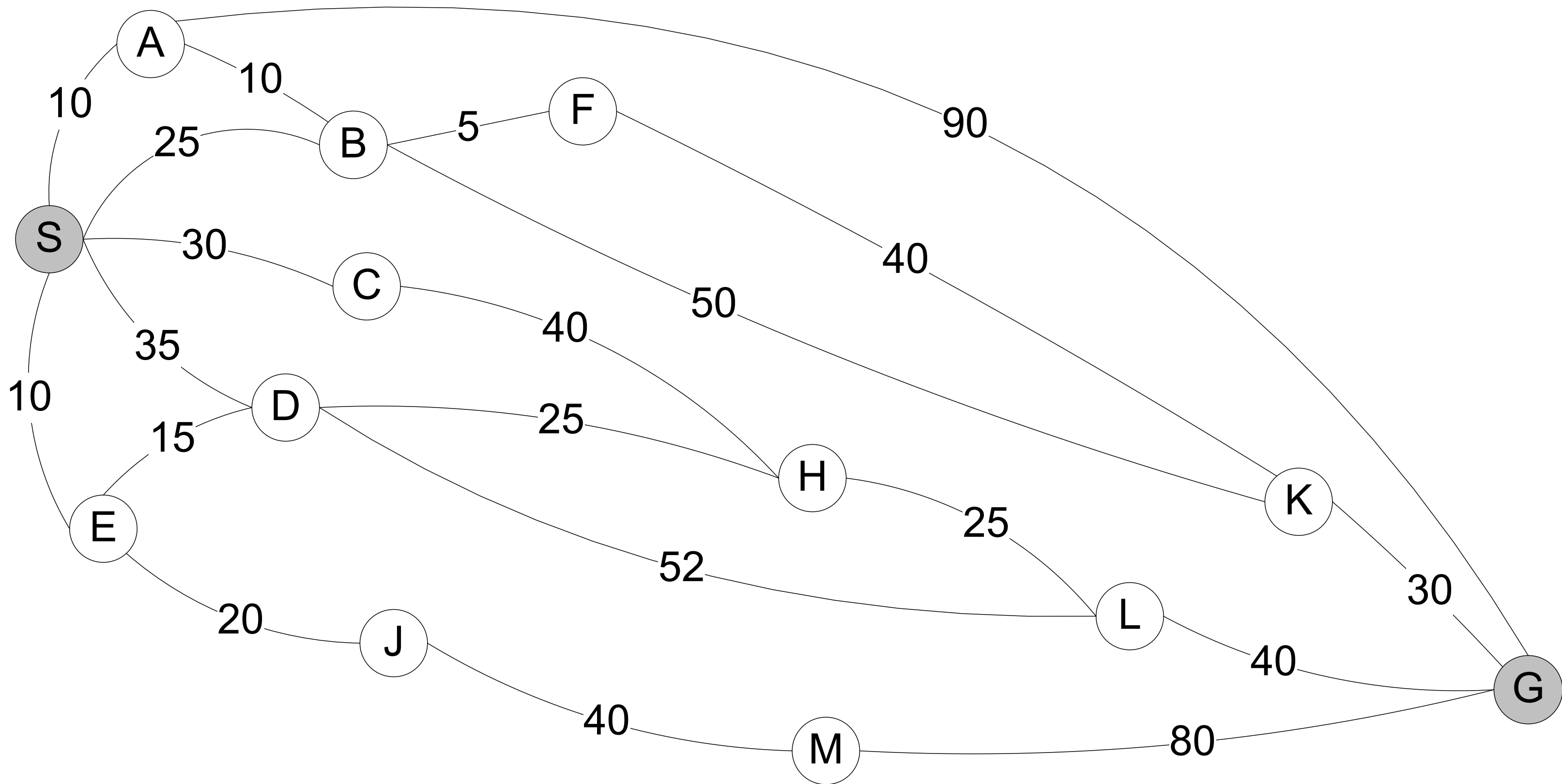
- *Complete* dan *Optimal*
- 

A*



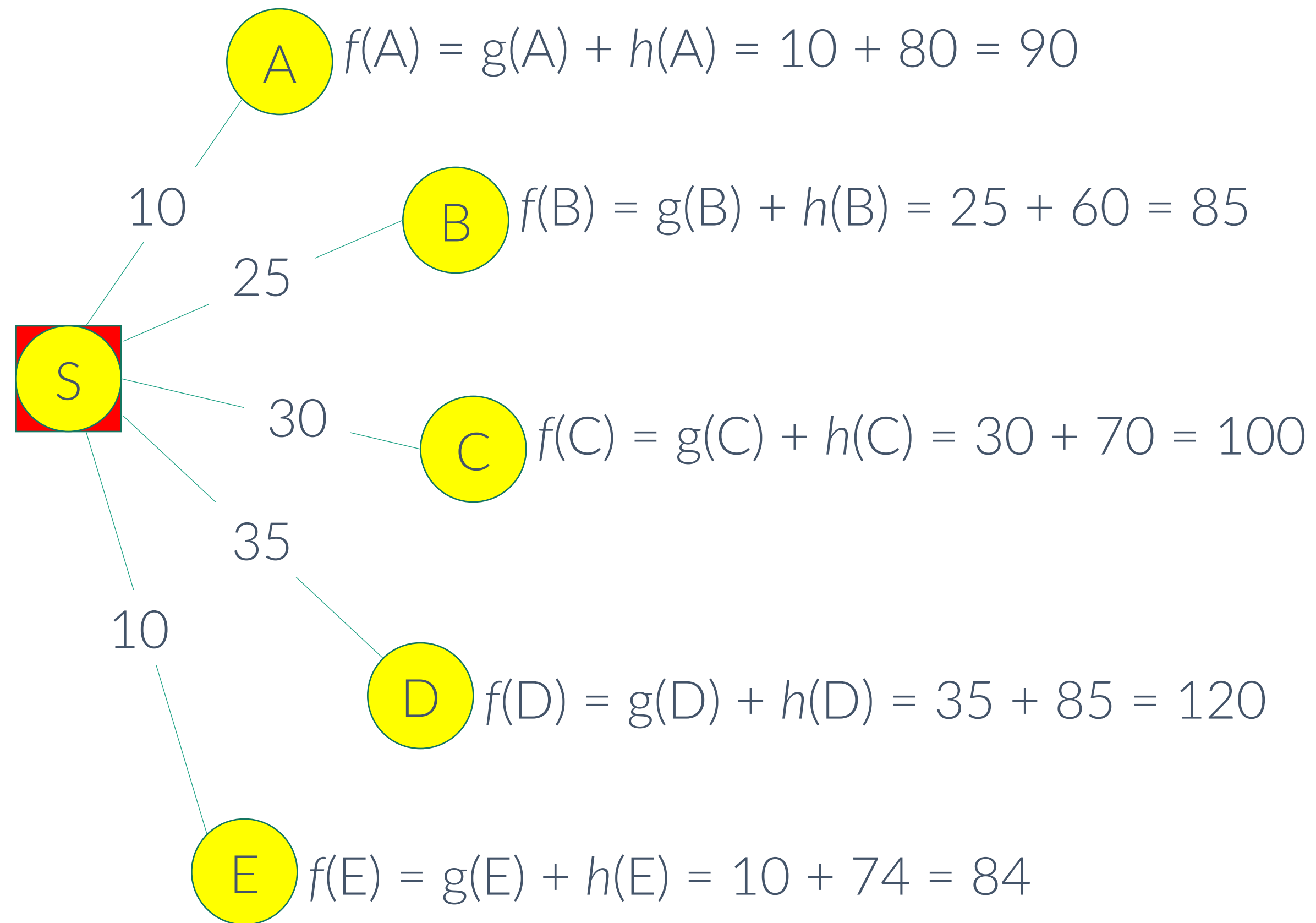
n	S	A	B	C	D	E	F	G	H	J	K	L	M
$h(n)$	80	80	60	70	85	74	70	0	40	100	30	20	70

A*

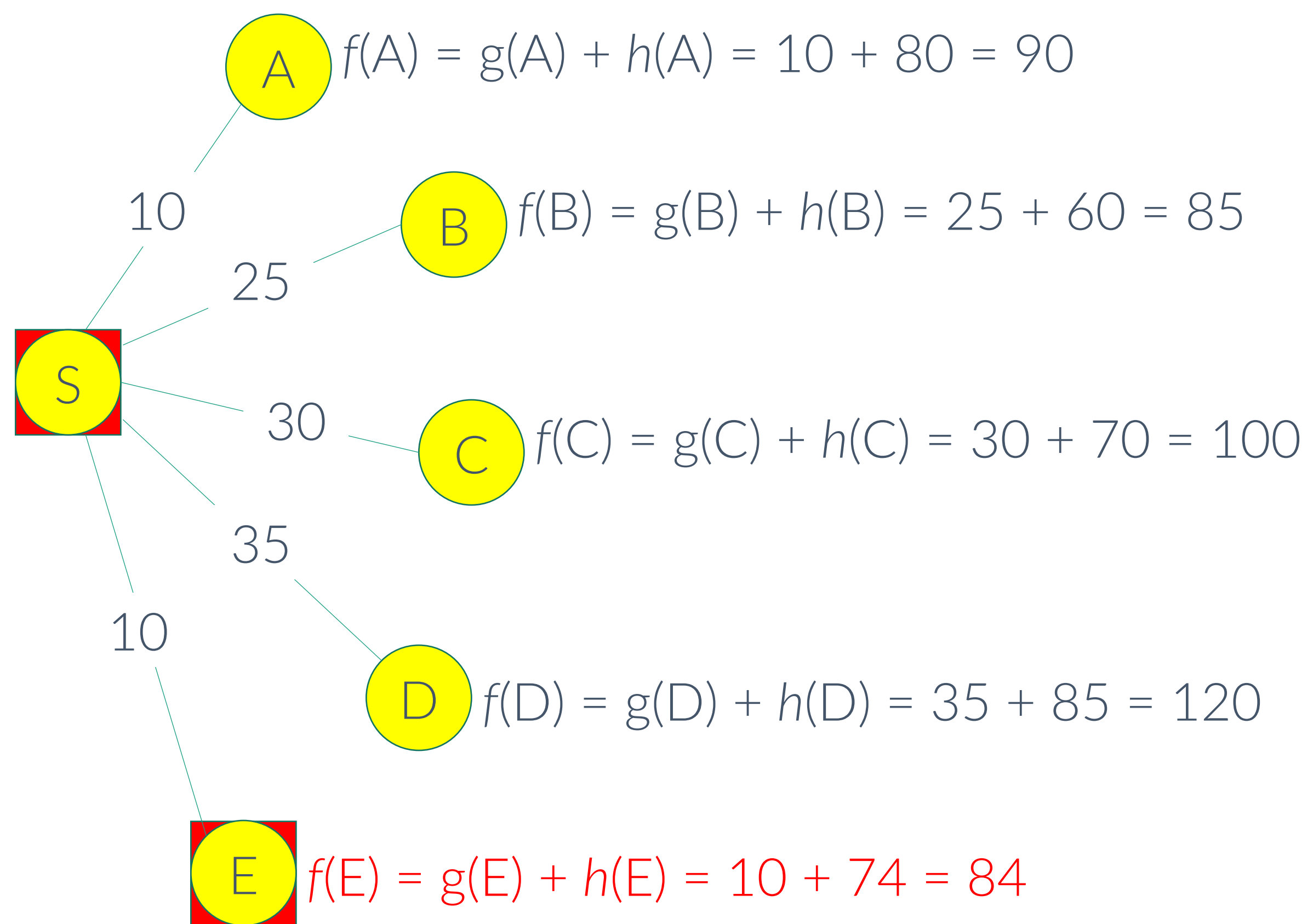


n	S	A	B	C	D	E	F	G	H	J	K	L	M
$h(n)$	80	80	60	70	85	74	70	0	40	100	30	20	70

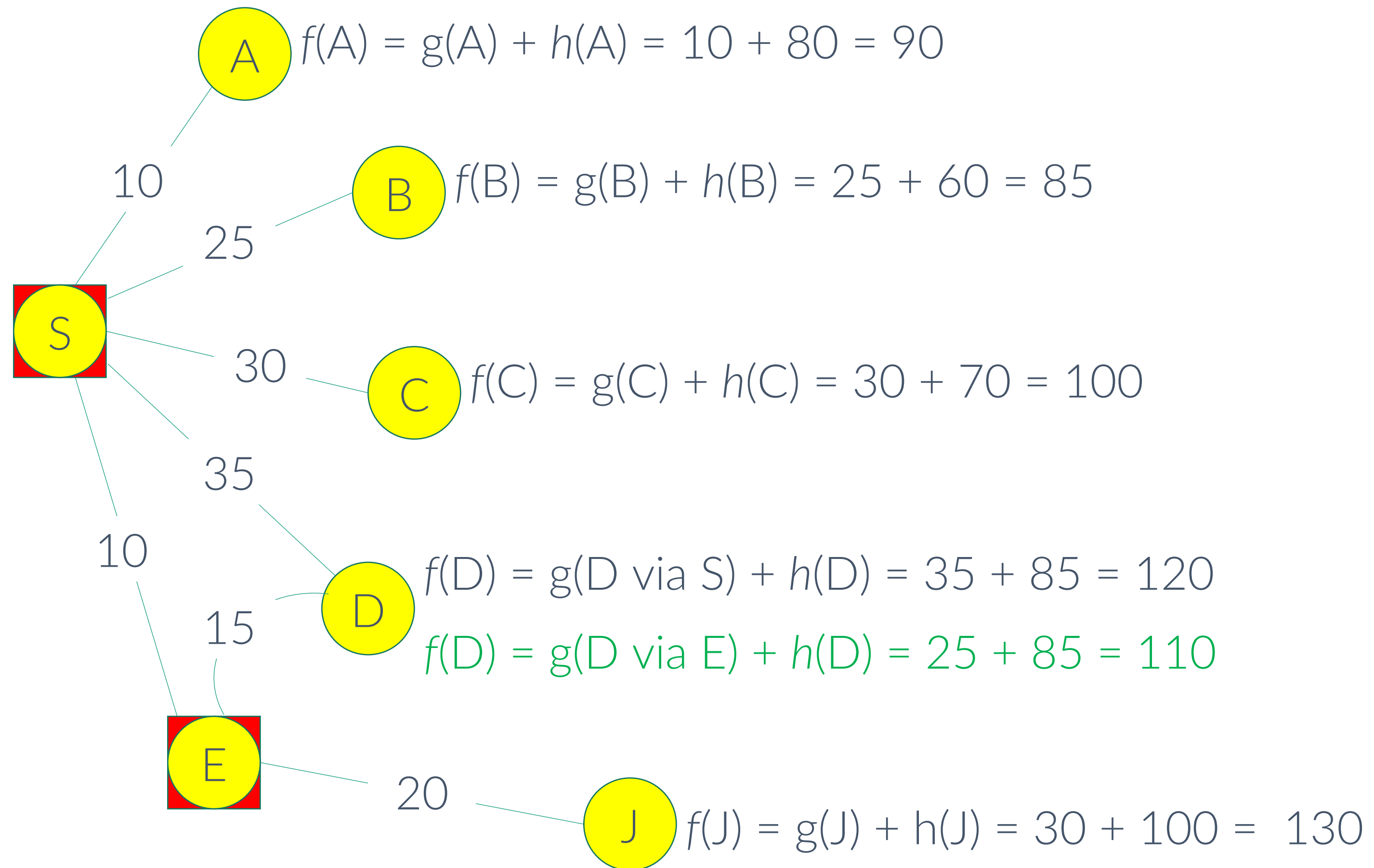
A^*



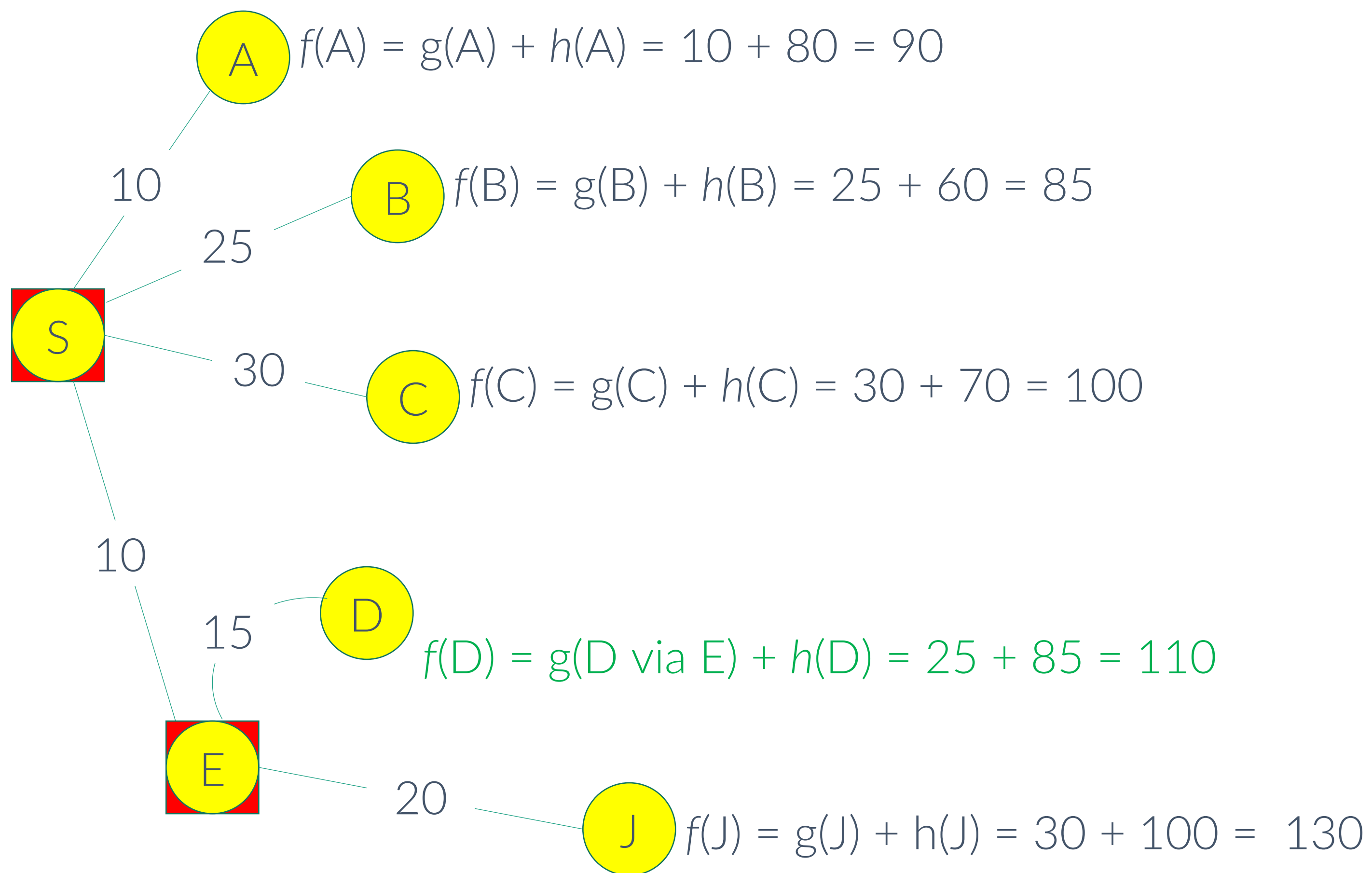
A^*



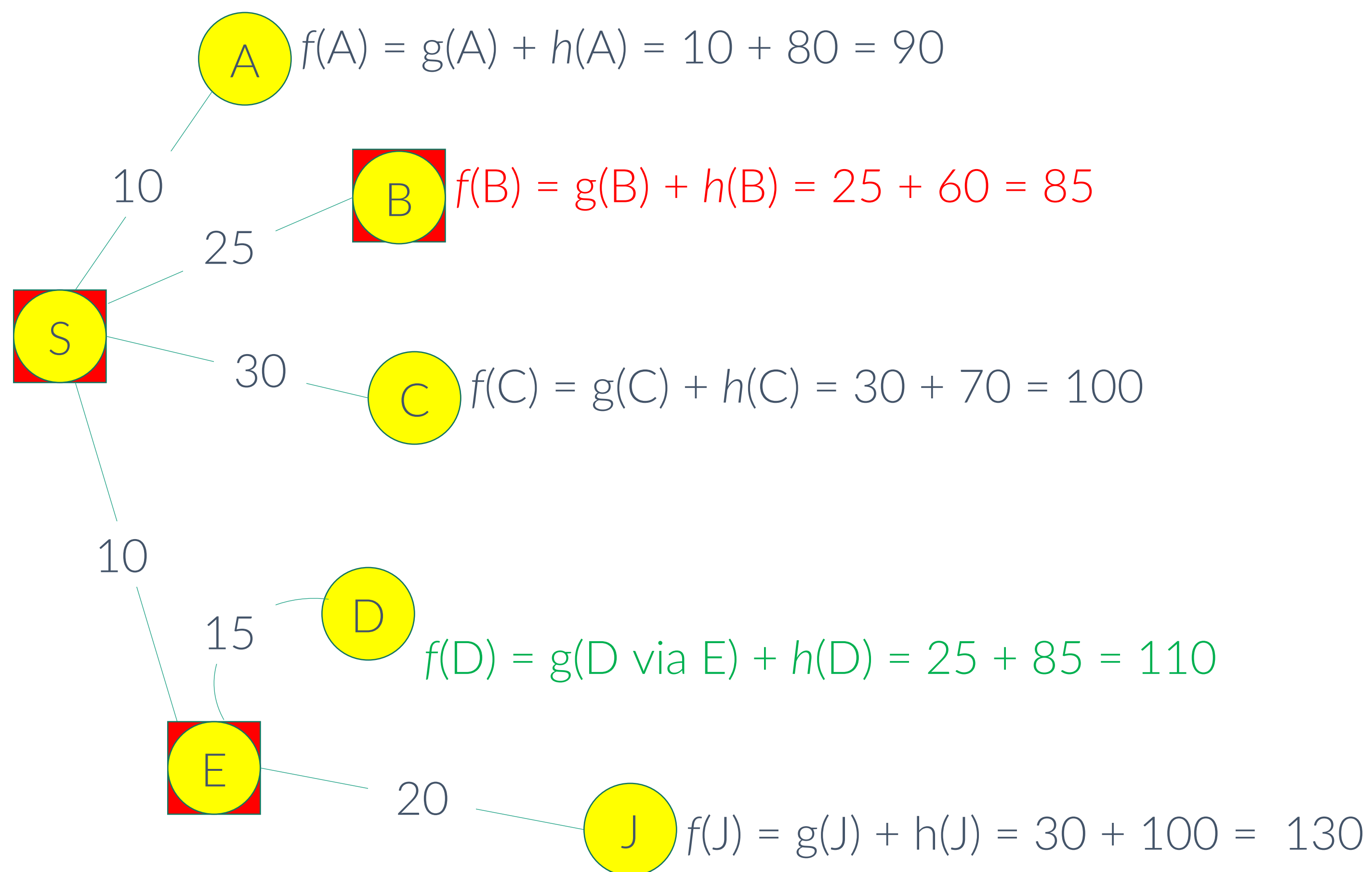
A^*



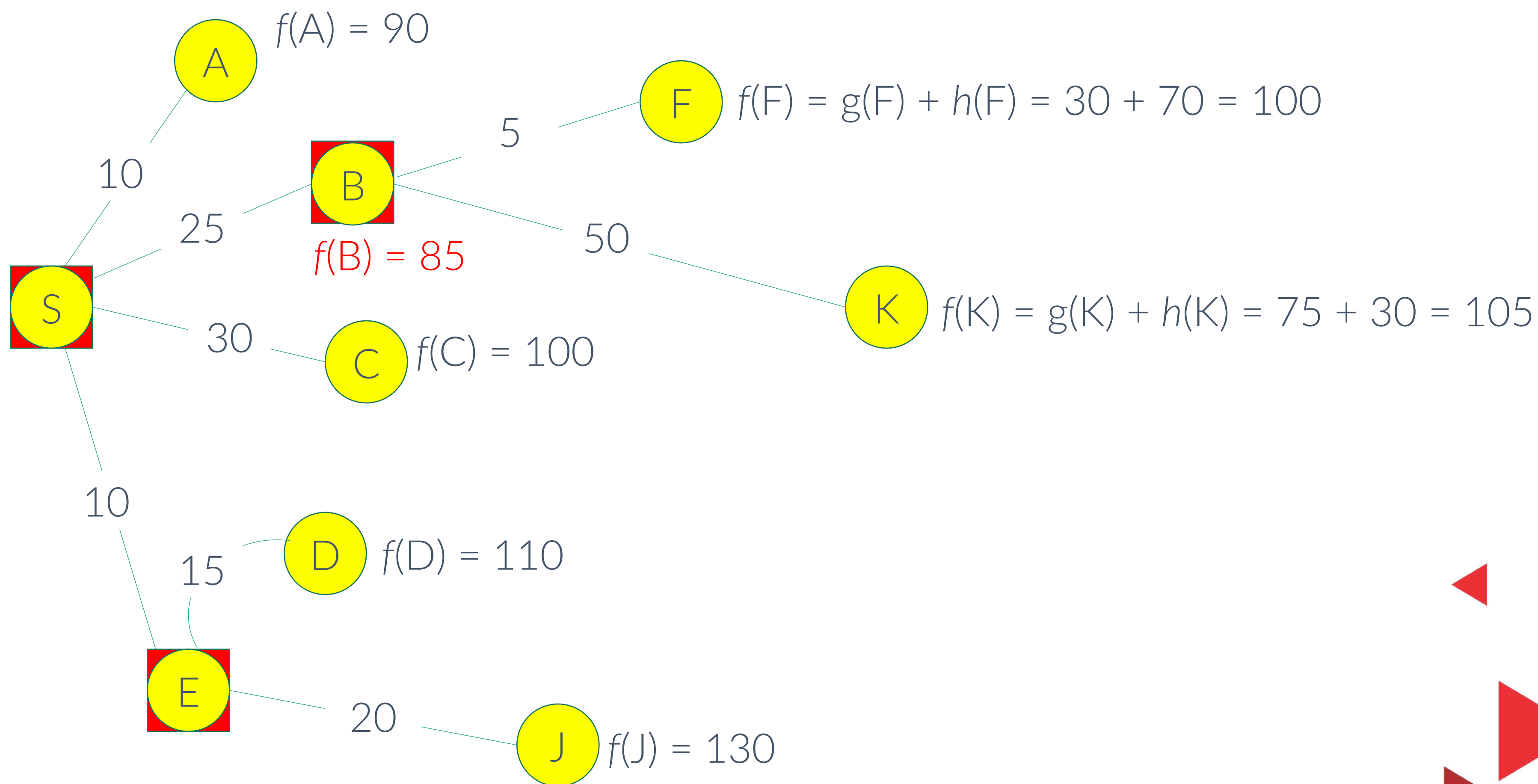
A^*



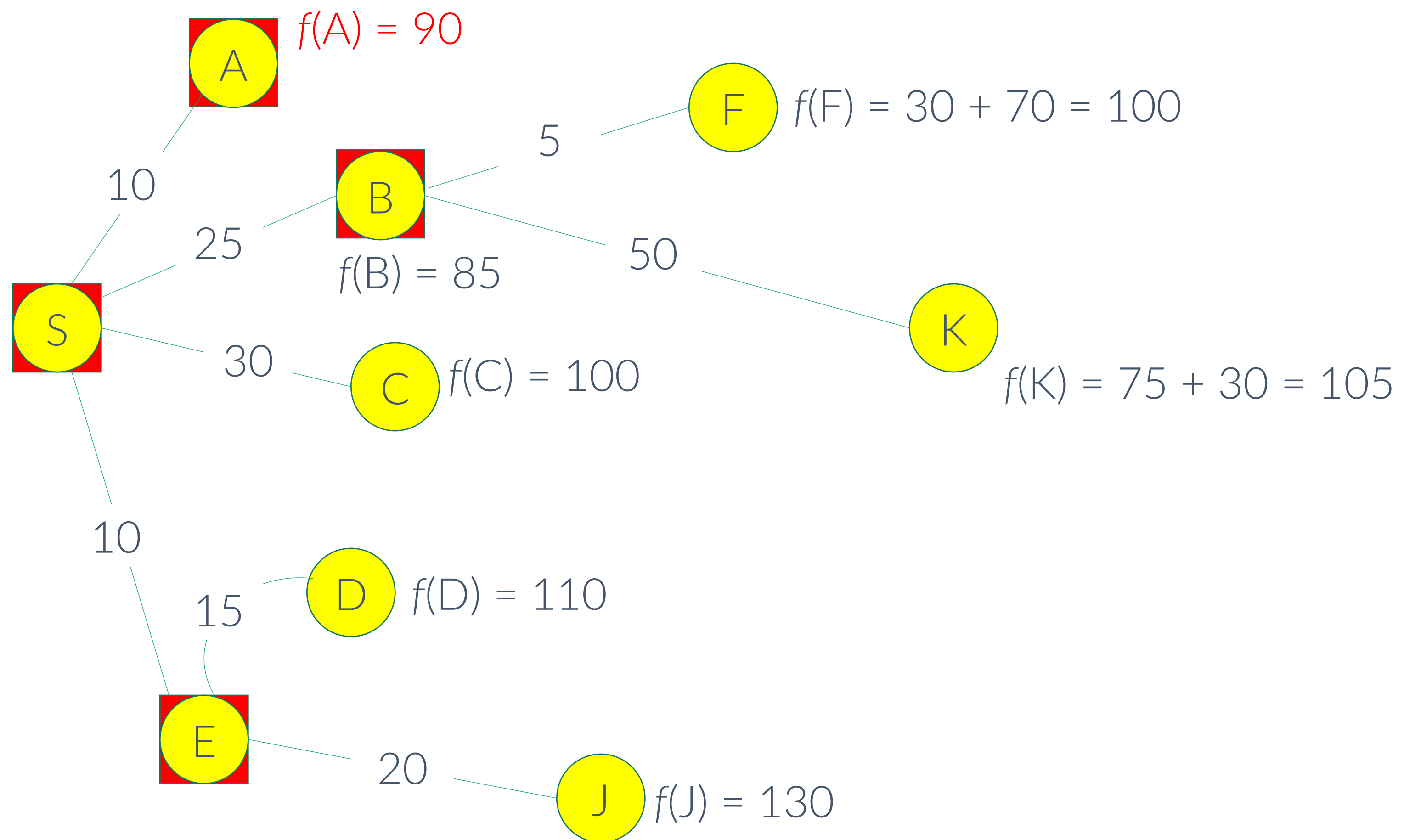
A^*



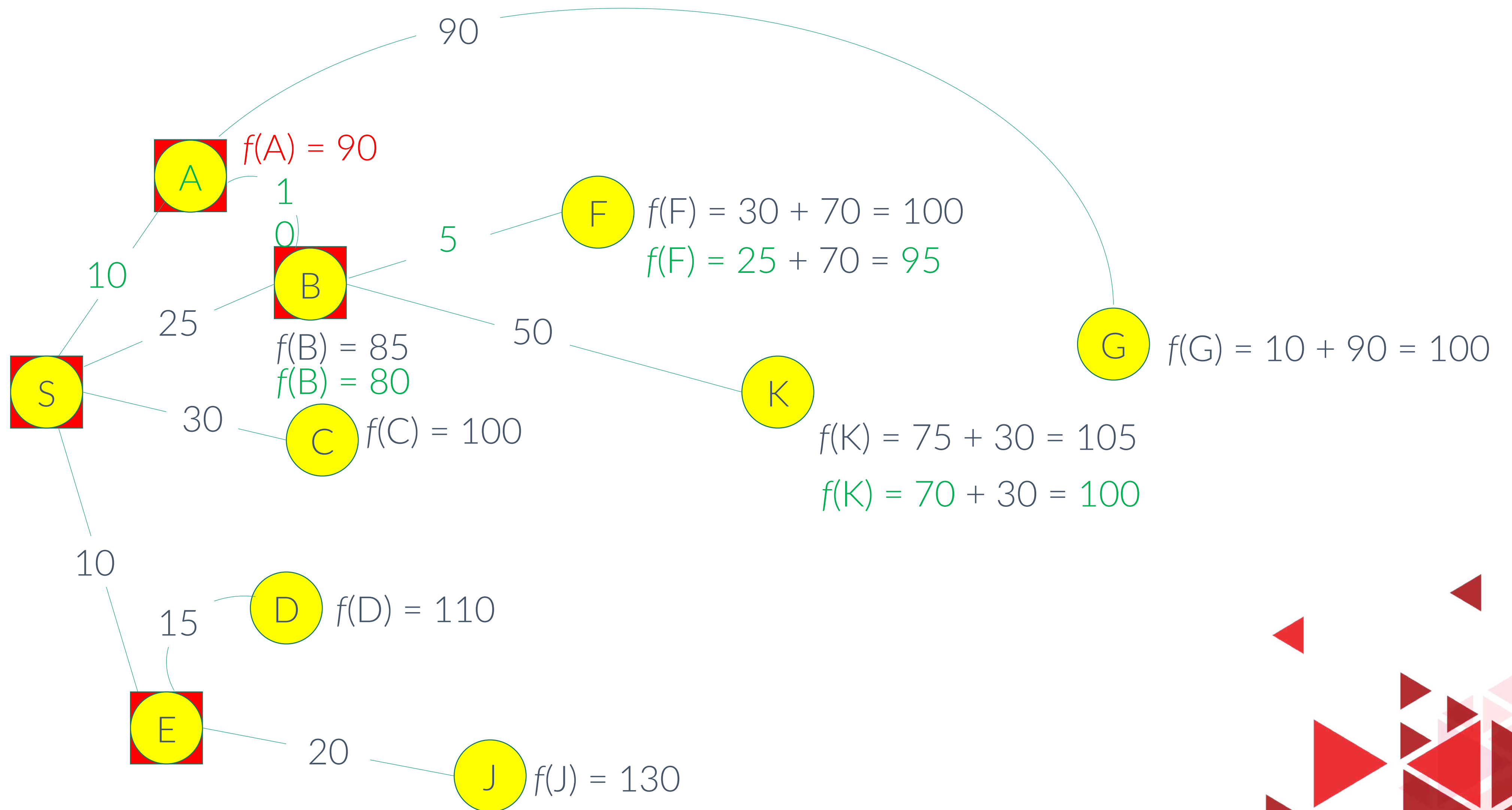
A^*



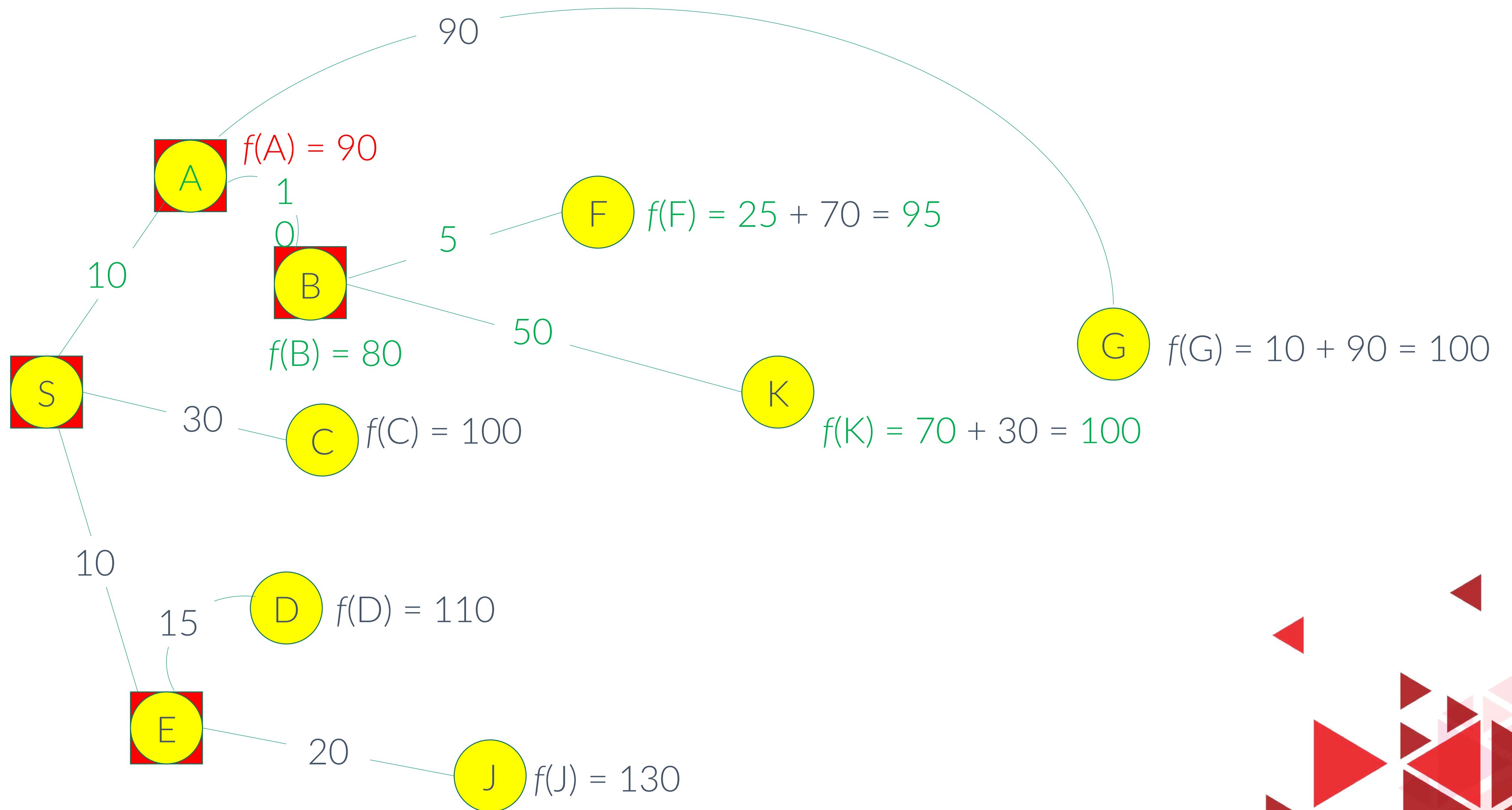
A^*



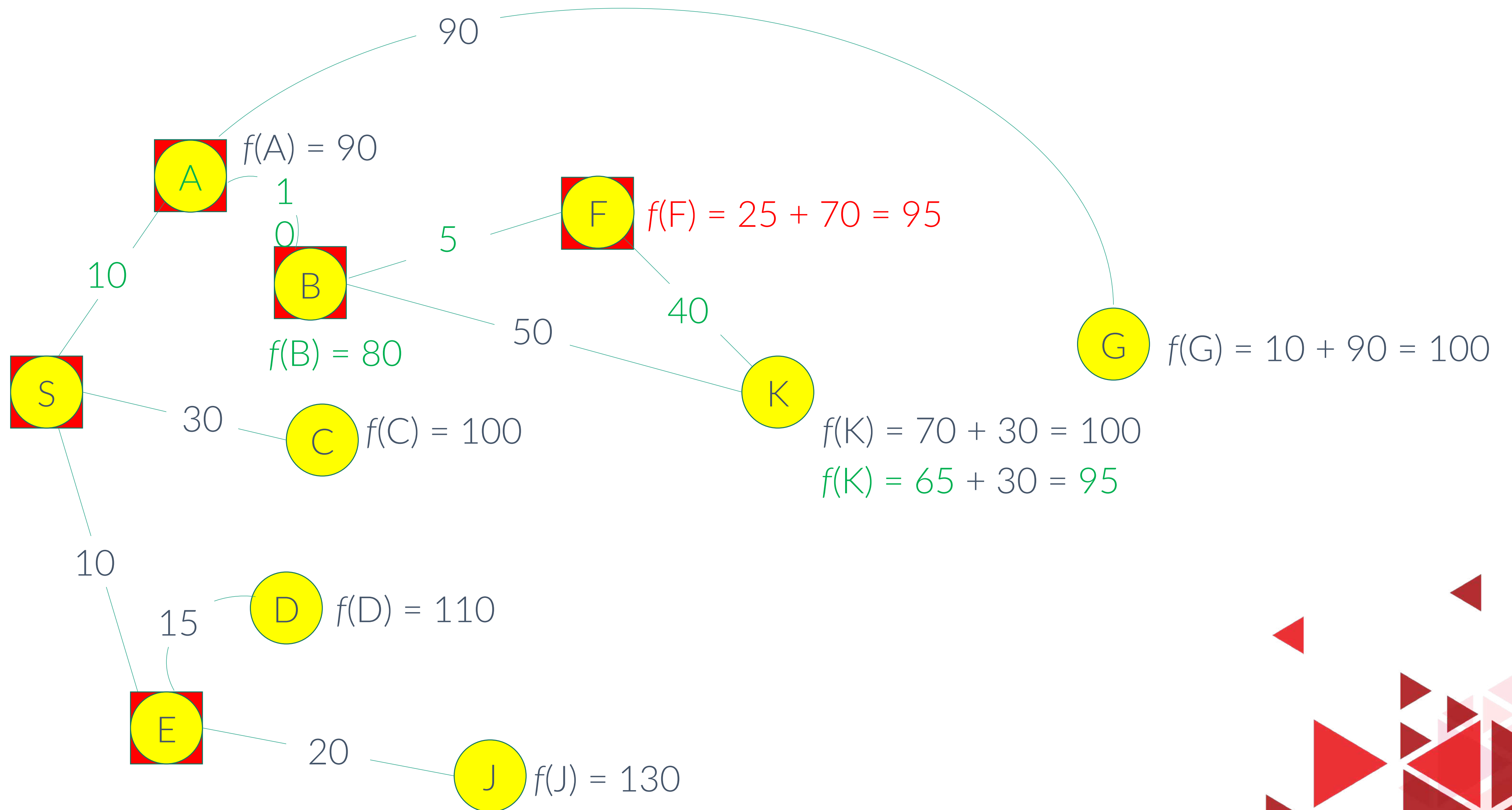
A^*



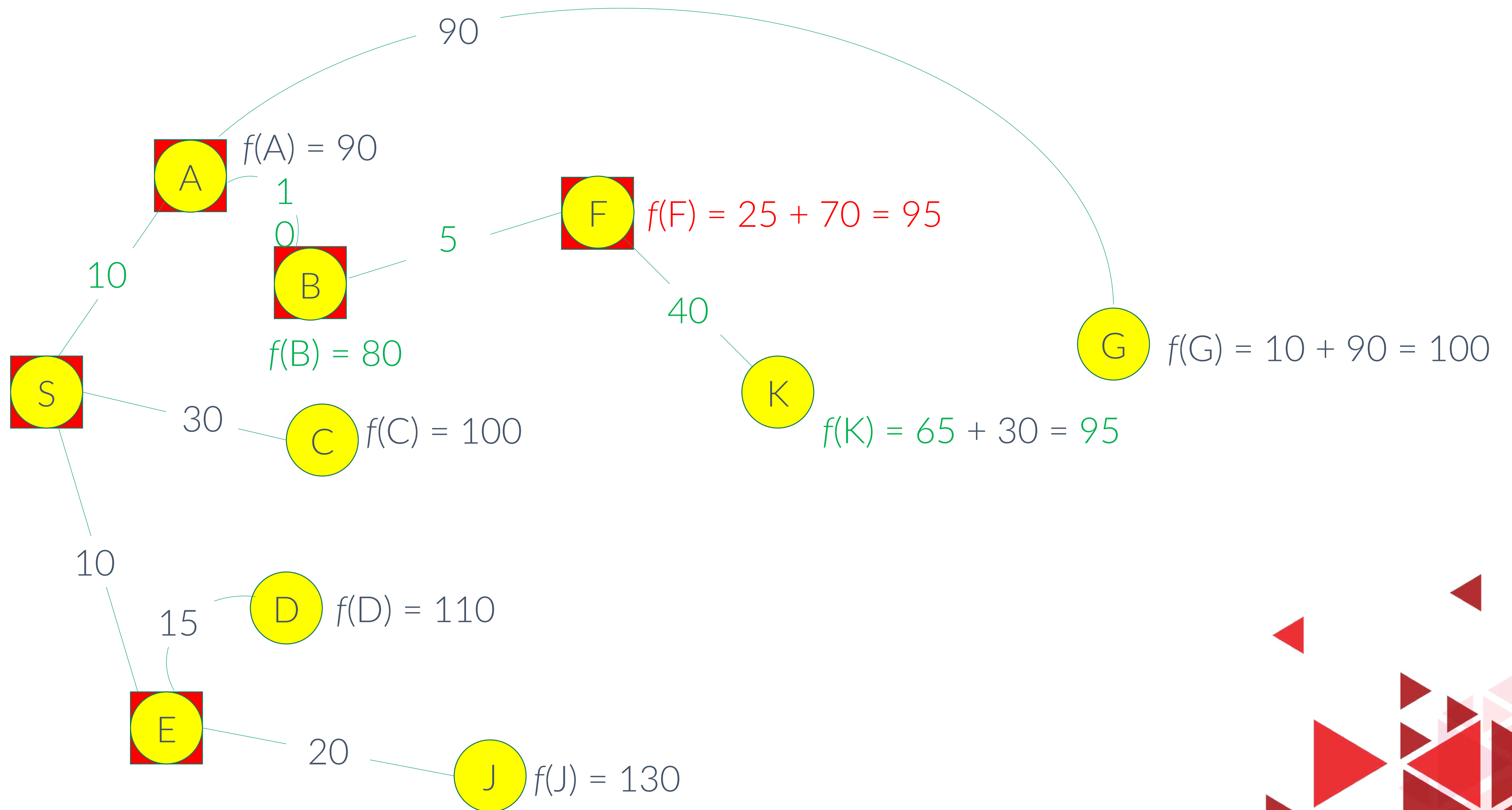
A^*



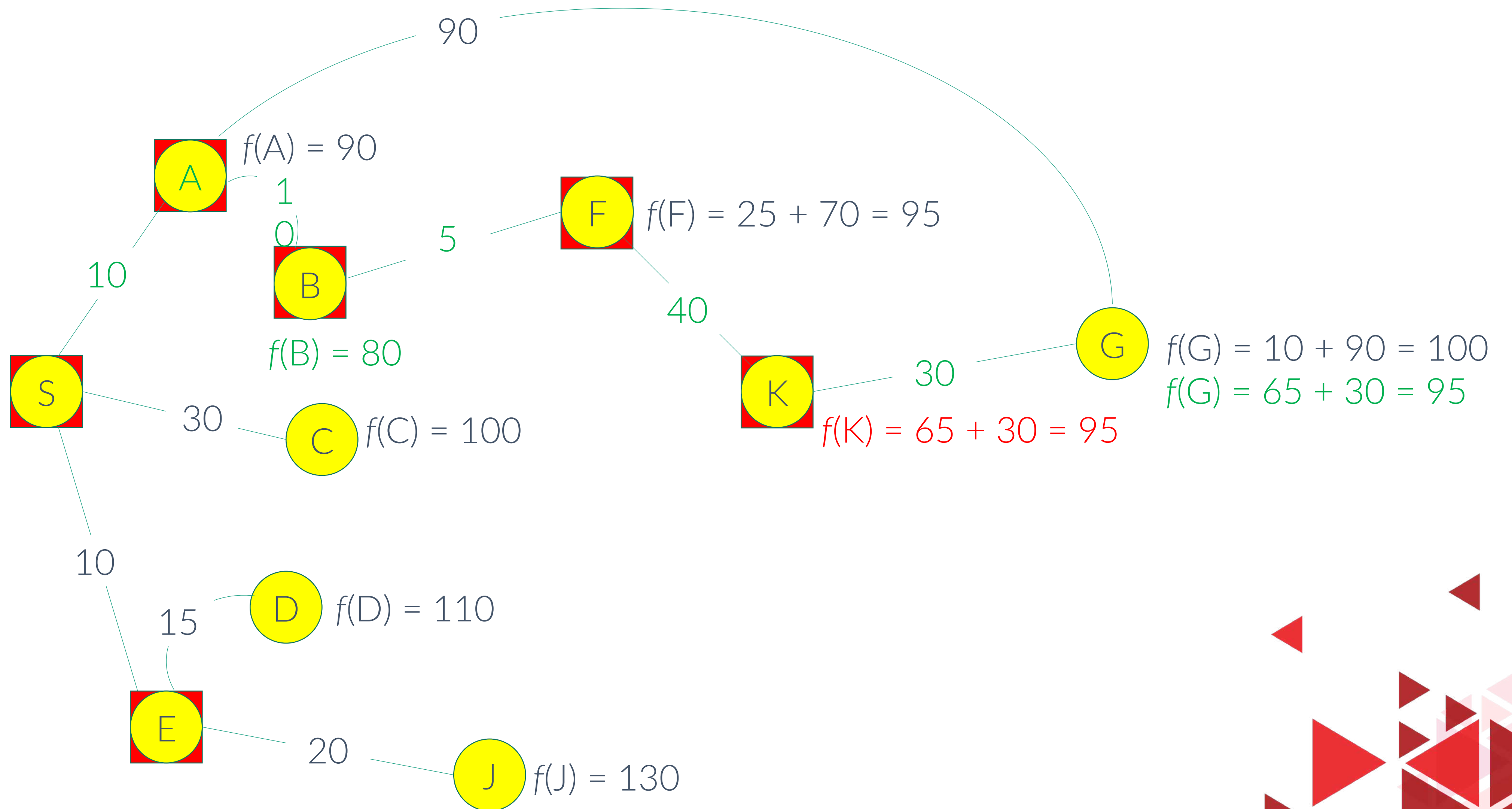
A^*



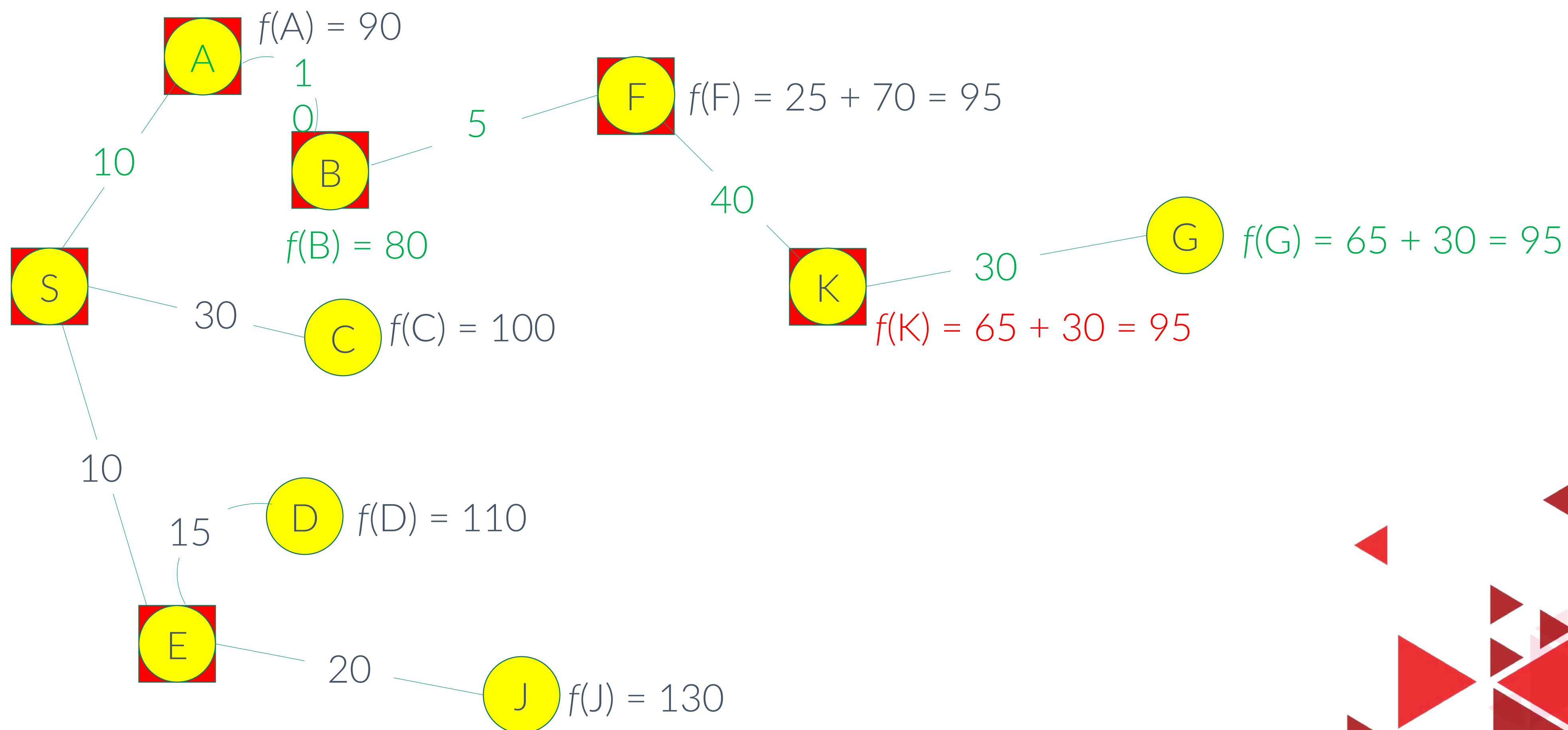
A^*



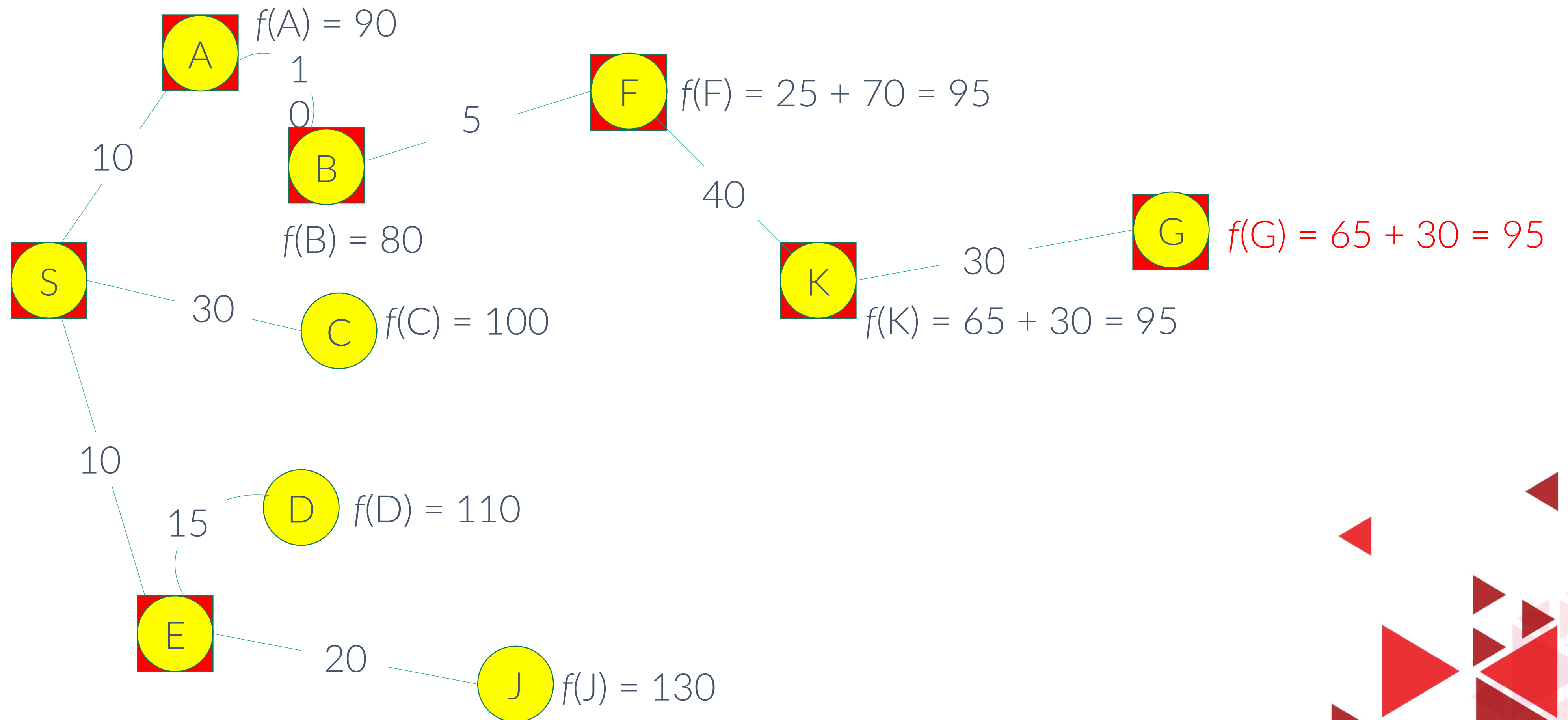
A^*



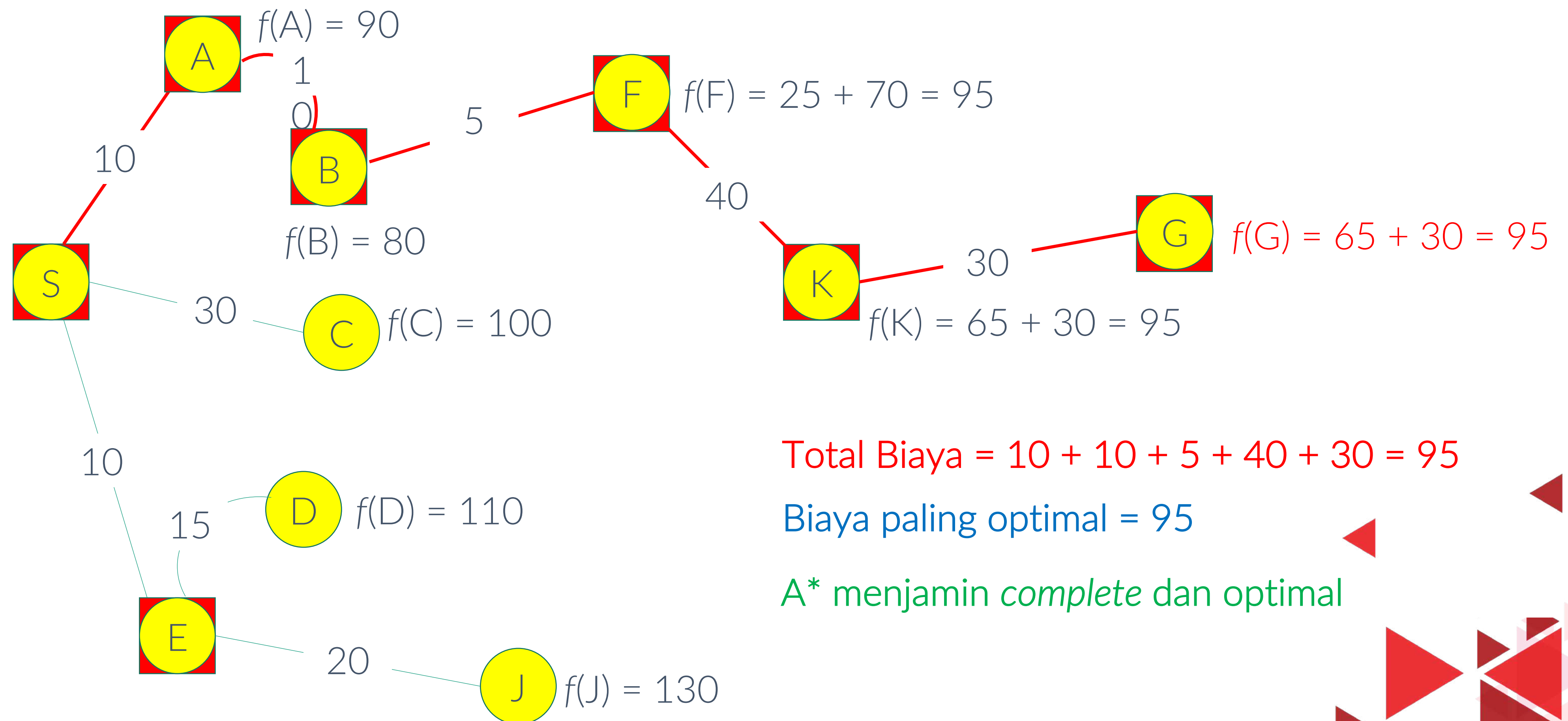
A^*



A^*



A*



CAK2HAB3 - Dasar Kecerdasan Artifisial

Iterative Deepening A*

S1 Informatika – Fakultas Informatika

Semester Genap 2024/2025





Iterative Deepening A*

- 
- Iteratif dengan batasan berupa f-limit
 - *Complete & Optimal*
 - *Time complexity* tinggi
 - *Space complexity* sangat rendah
 - Sesuai untuk *low memory-devices*
- 

function IDA*(*masalah*) **returns** *solusi*
inputs: *masalah*, sebuah masalah
local variables: *f-limit*, batasan *f*-Cost saat ini
root, sebuah simpul

root $\leftarrow$ MAKE-SIMPUL(INITIAL-STATE[*masalah*])
f-limit $\leftarrow$ *f*-Cost(*root*)

loop do

solusi, *f-limit* $\leftarrow$ DFS-CONTOUR(*root*, *f-limit*)

if *solusi* bukan *null* **then** return *solusi*

if *f-limit* sama dengan INFINITE **then** return gagal

end

function DFS-CONTOUR(*simpul*, *f-limit*) **returns** *solusi*, batasan *f*-Cost yang baru

inputs: *simpul*, sebuah simpul

f-limit, batasan *f*-Cost saat ini

local variables: *next-f*, batasan *f*-Cost untuk penelusuran berikutnya, pada awalnya variabel ini di beri nilai tak hingga (INFINITE)

if *f*-Cost [*simpul*] > *f-limit* **then return** *null*, *f*-Cost[*simpul*]

if GOAL-TEST[problem] (STATE[*simpul*]) **then return** *simpul*, *f-limit*

for setiap simpul *s* yang menjadi SUCCESSOR(*simpul*) **do**

solusi, *new-f* $\leftarrow$ DFS-CONTOUR(*s*, *f-limit*)

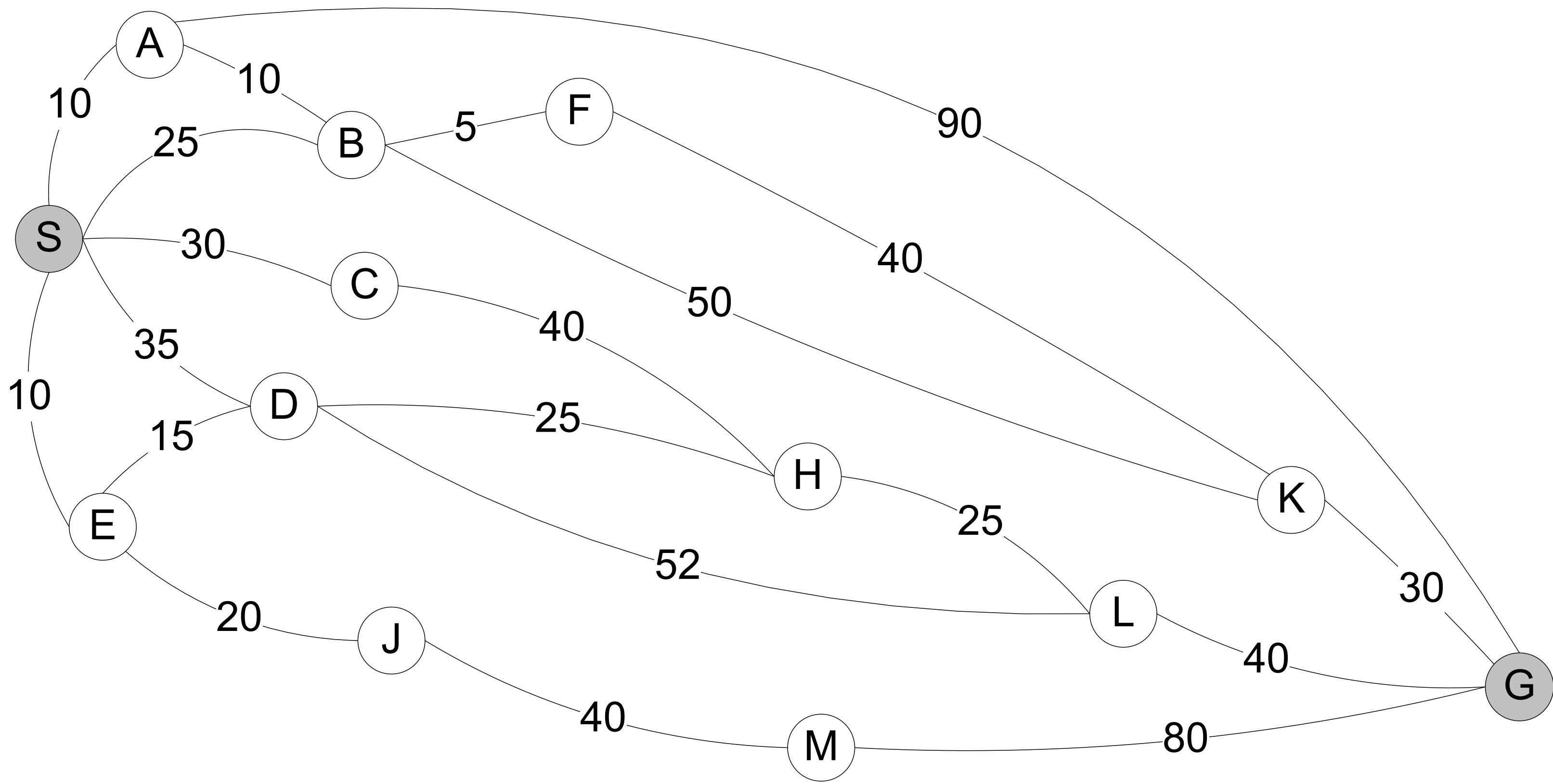
if *solusi* bukan *null* **then return** *solusi*, *f-limit*

next-f $\leftarrow$ MIN(*next-f*, *new-f*)

end

return *null*, *next-f*

Iterative Deepening A*



n	S	A	B	C	D	E	F	G	H	J	K	L	M
$h(n)$	80	80	60	70	85	74	70	0	40	100	30	20	70

Kondisi awal, next-f = infinity

Input:

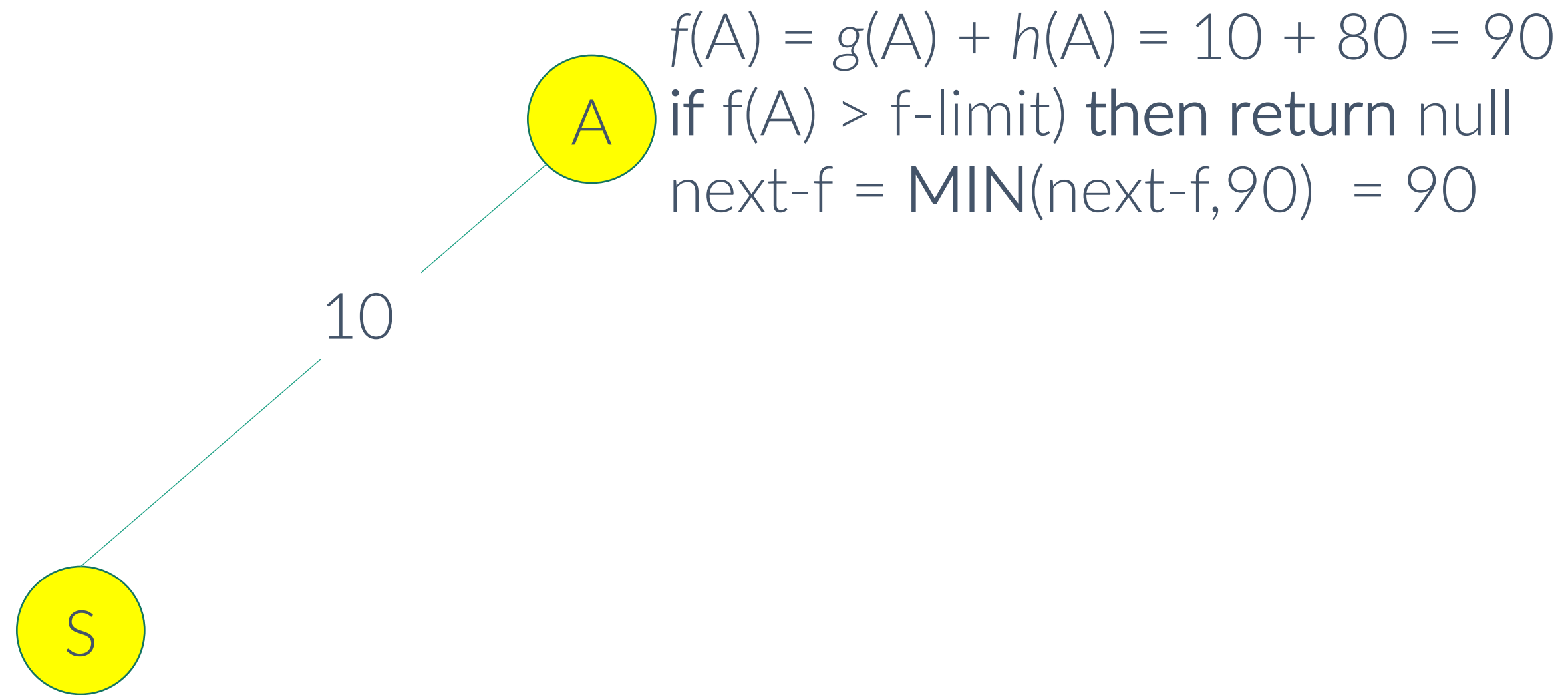
node = S

f-limit = 80

Return:

null

next-f = 90



next-f = 90

Input:

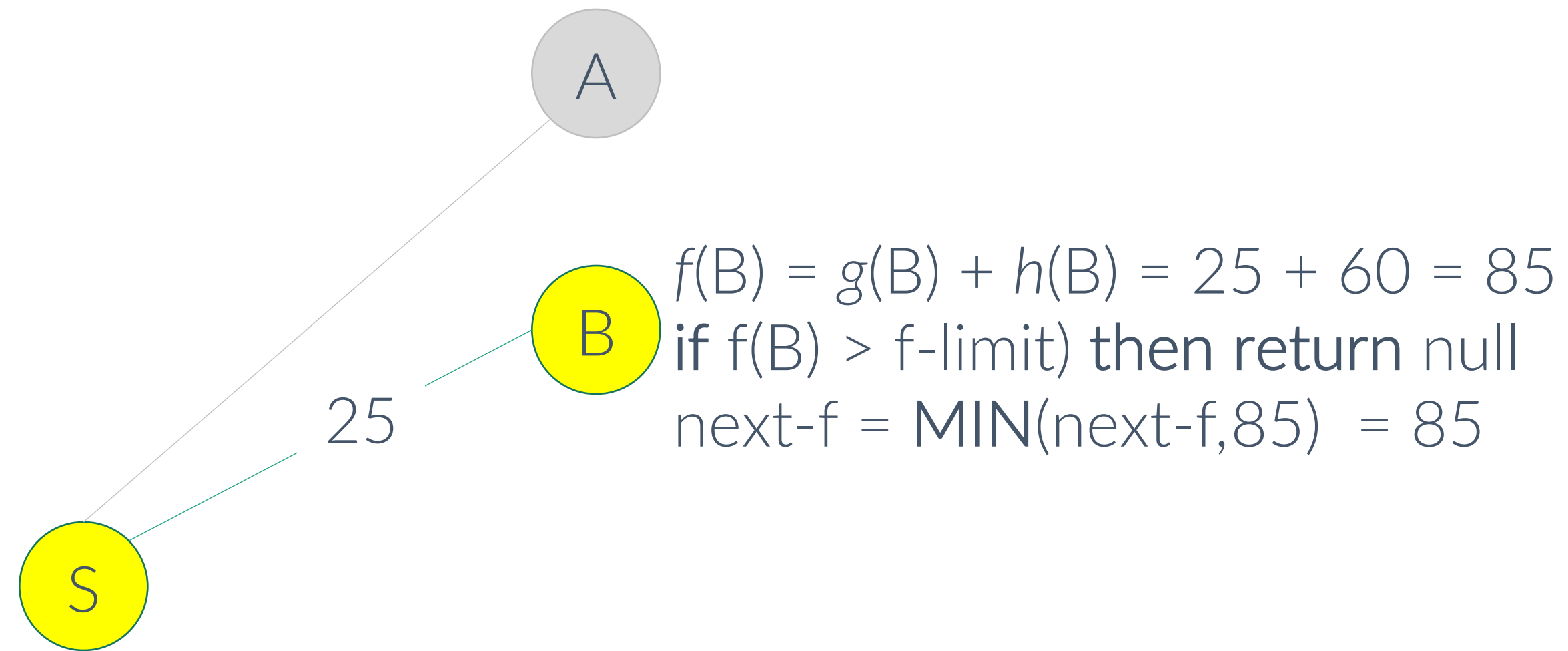
node = S

f-limit = 80

Return:

null

next-f = 85



next-f = 85

Input:

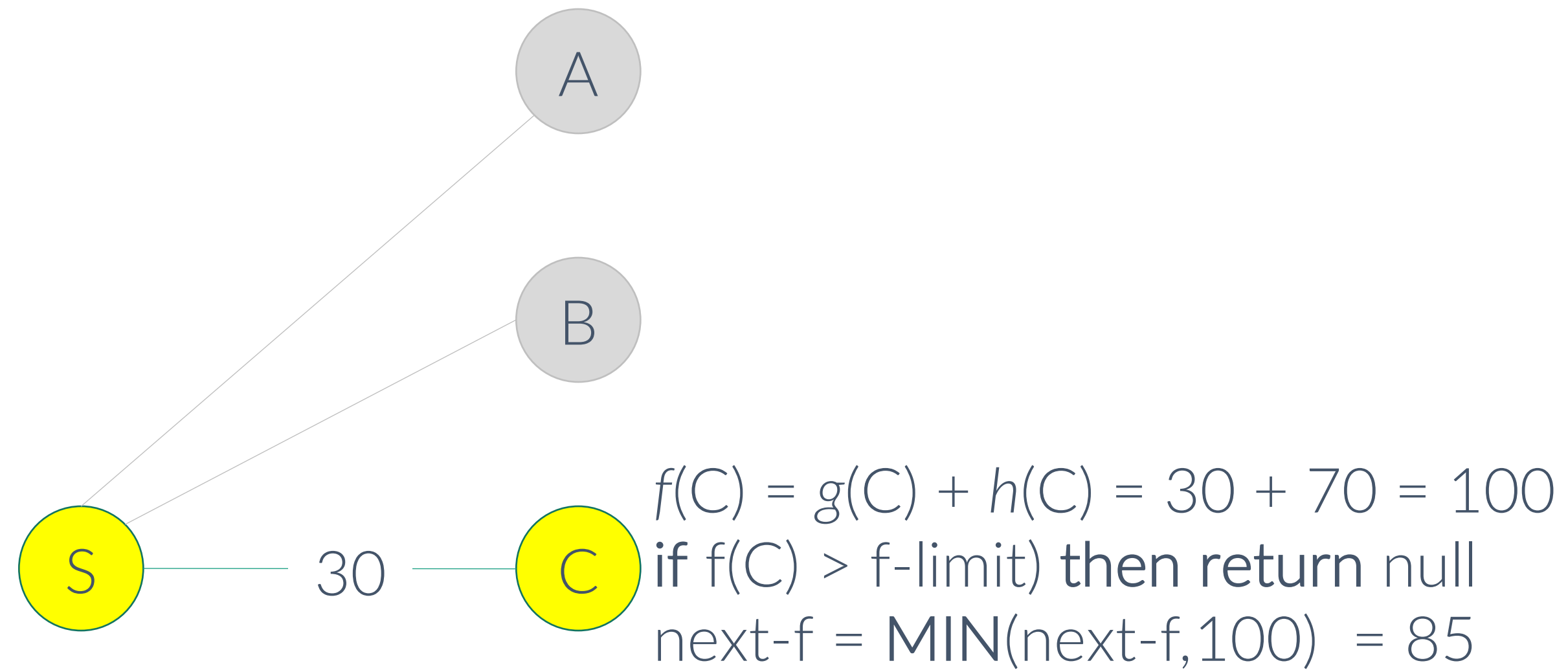
node = S

f-limit = 80

Return:

null

next-f = 85



next-f = 85

Input:

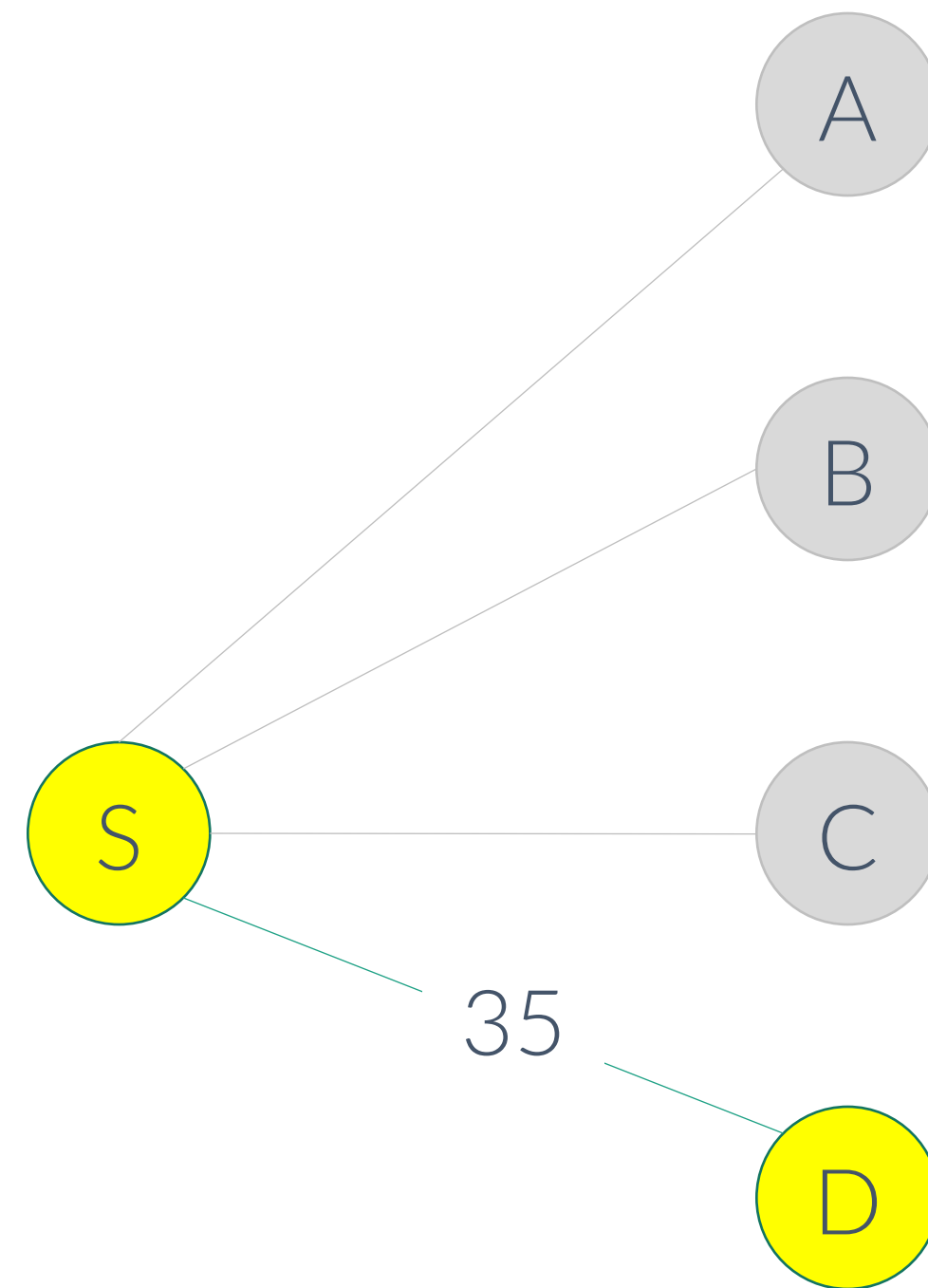
node = S

f-limit = 80

Return:

null

next-f = 85



$$f(D) = g(D) + h(D) = 35 + 85 = 120$$

if $f(D) > f\text{-limit}$ then return null

$$\text{next-f} = \text{MIN}(\text{next-f}, 120) = 85$$

next-f = 85

Input:

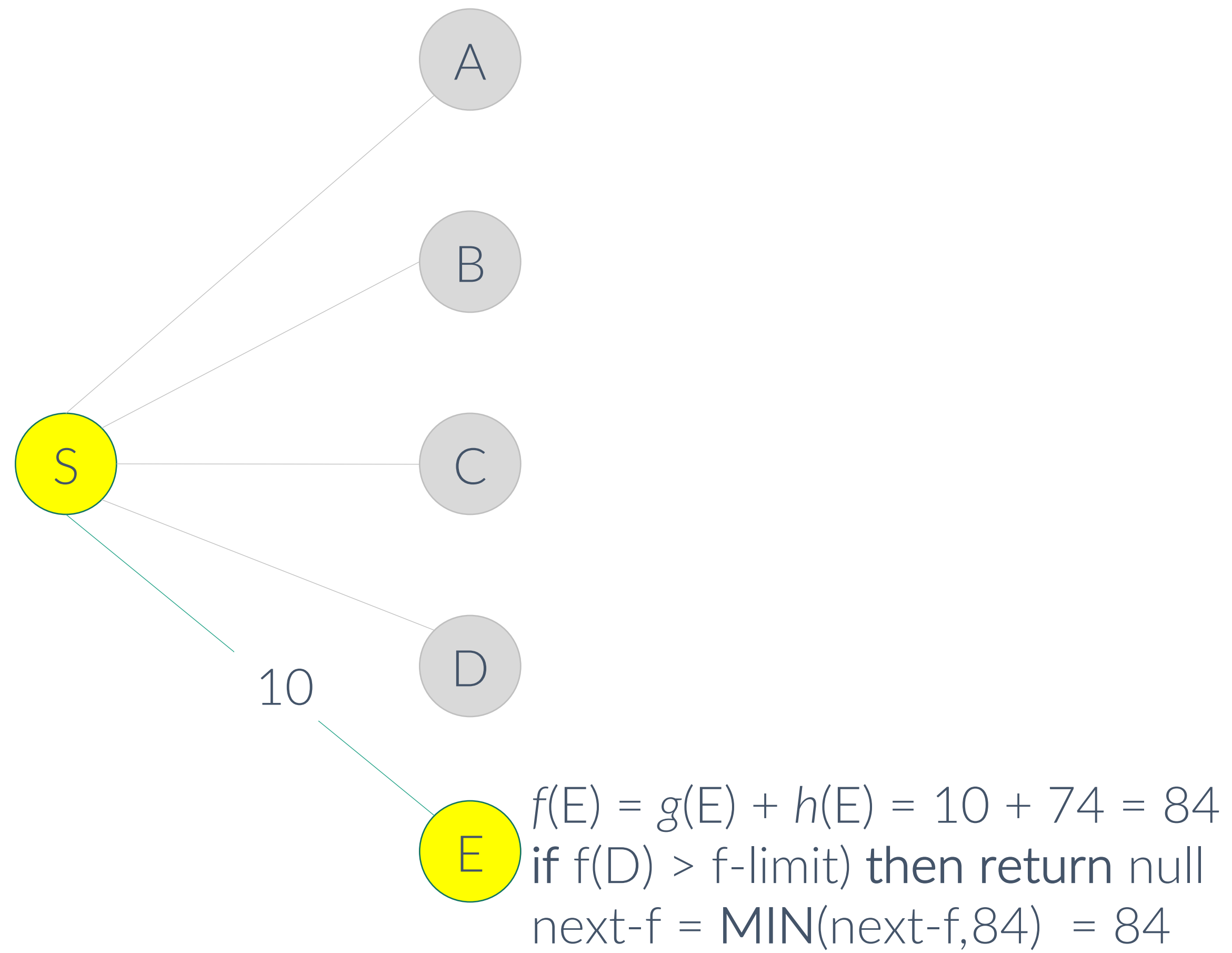
node = S

f-limit = 80

Return:

null

next-f = 84



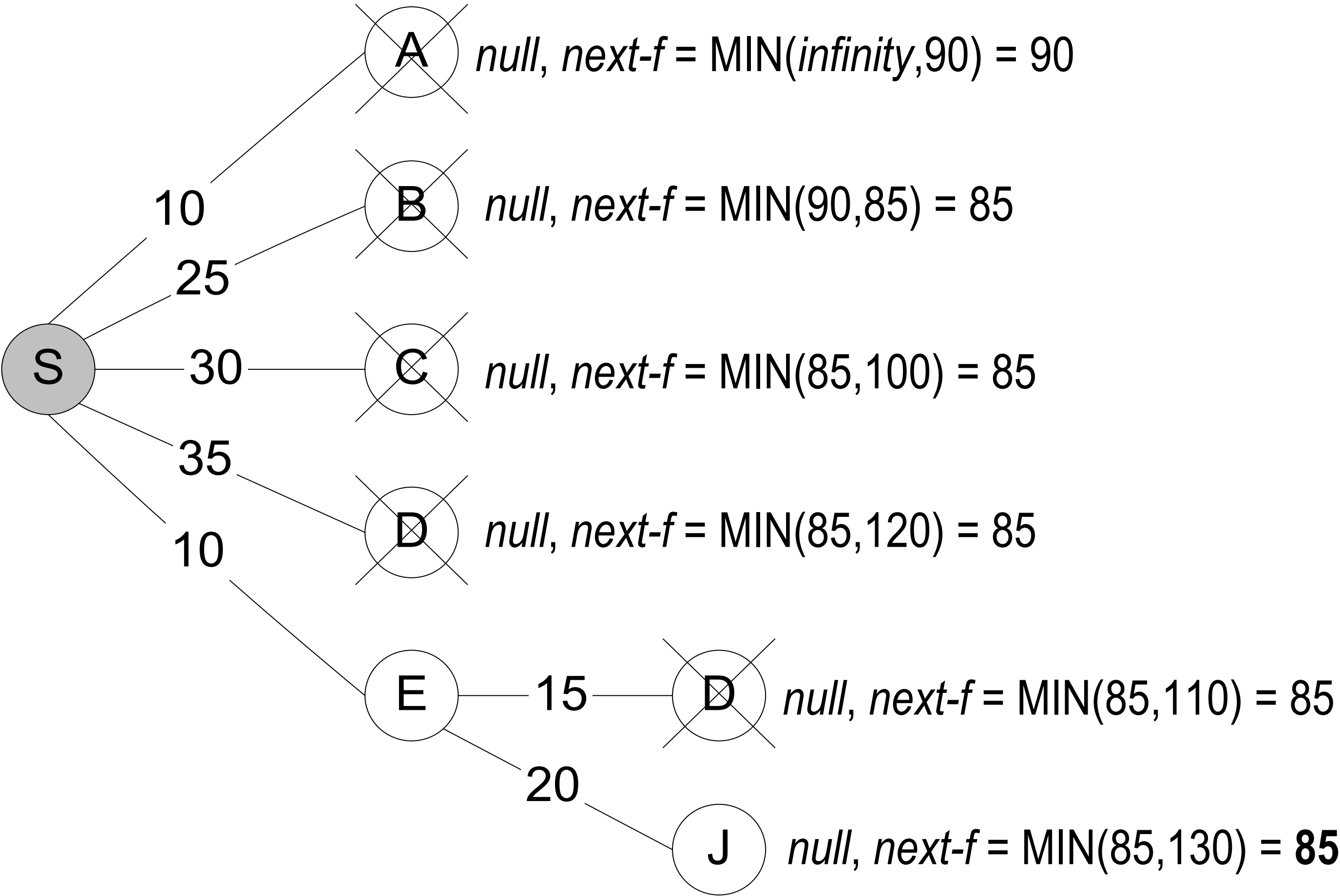
Langkah 2:

inputs:

- *node* = S
- *f-limit* = 84

returns:

- *null*
- *next-f* = 85



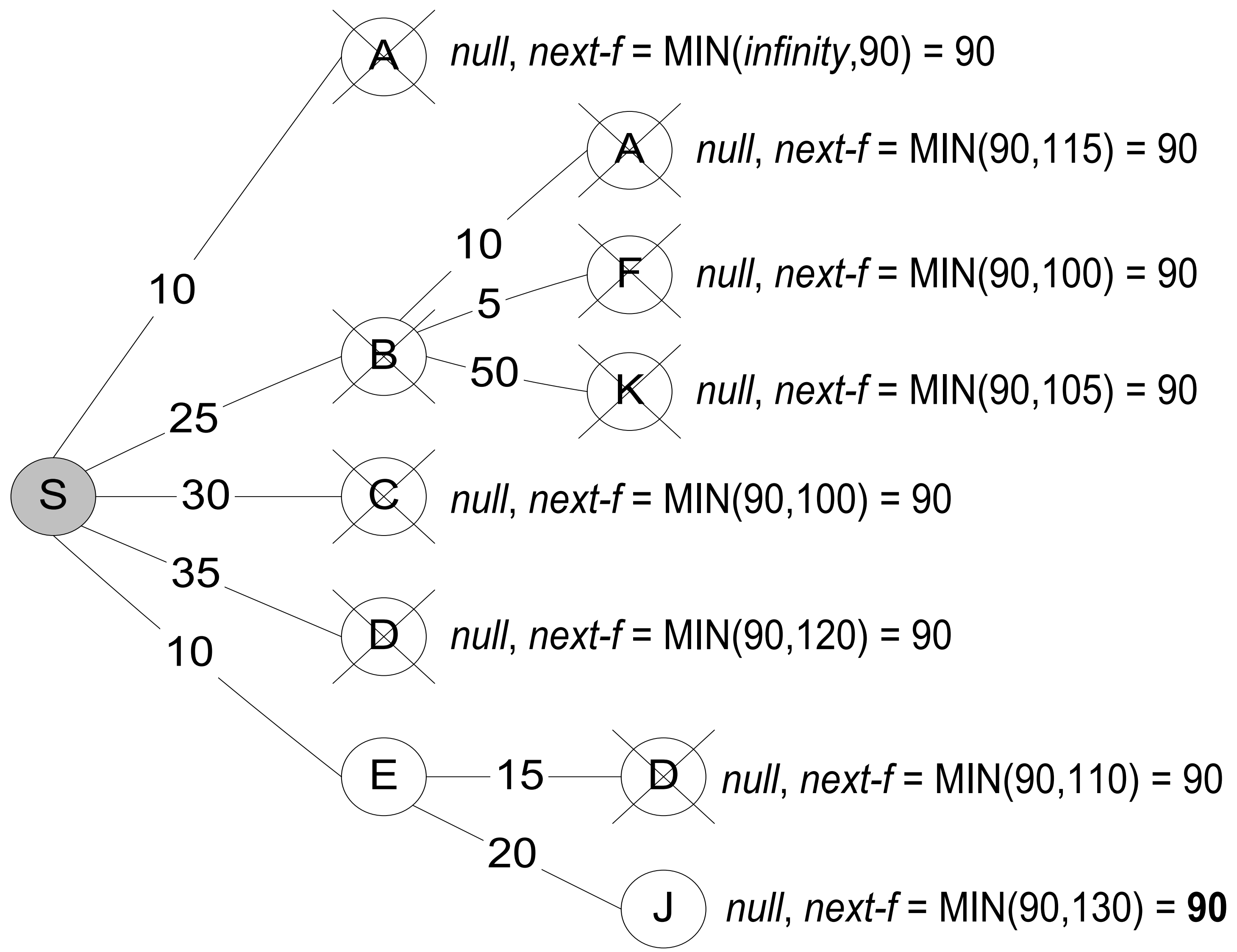
Langkah 3:

inputs:

- *node* = S
- *f-limit* = 85

returns:

- *null*
- *next-f* = 90



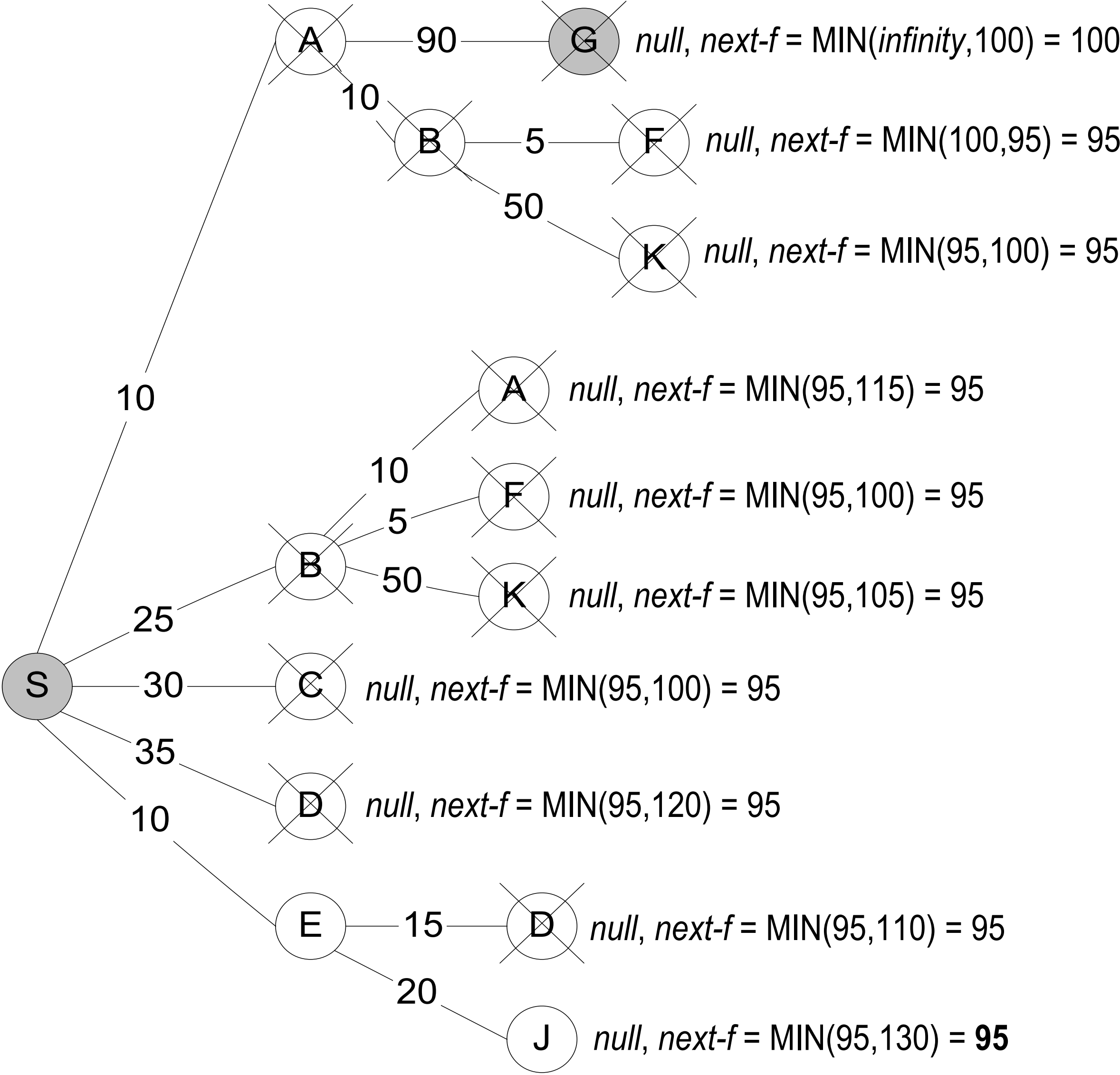
Langkah 4:

inputs:

- *node* = S
- *f-limit* = 90

returns:

- *null*
- *next-f* = 95



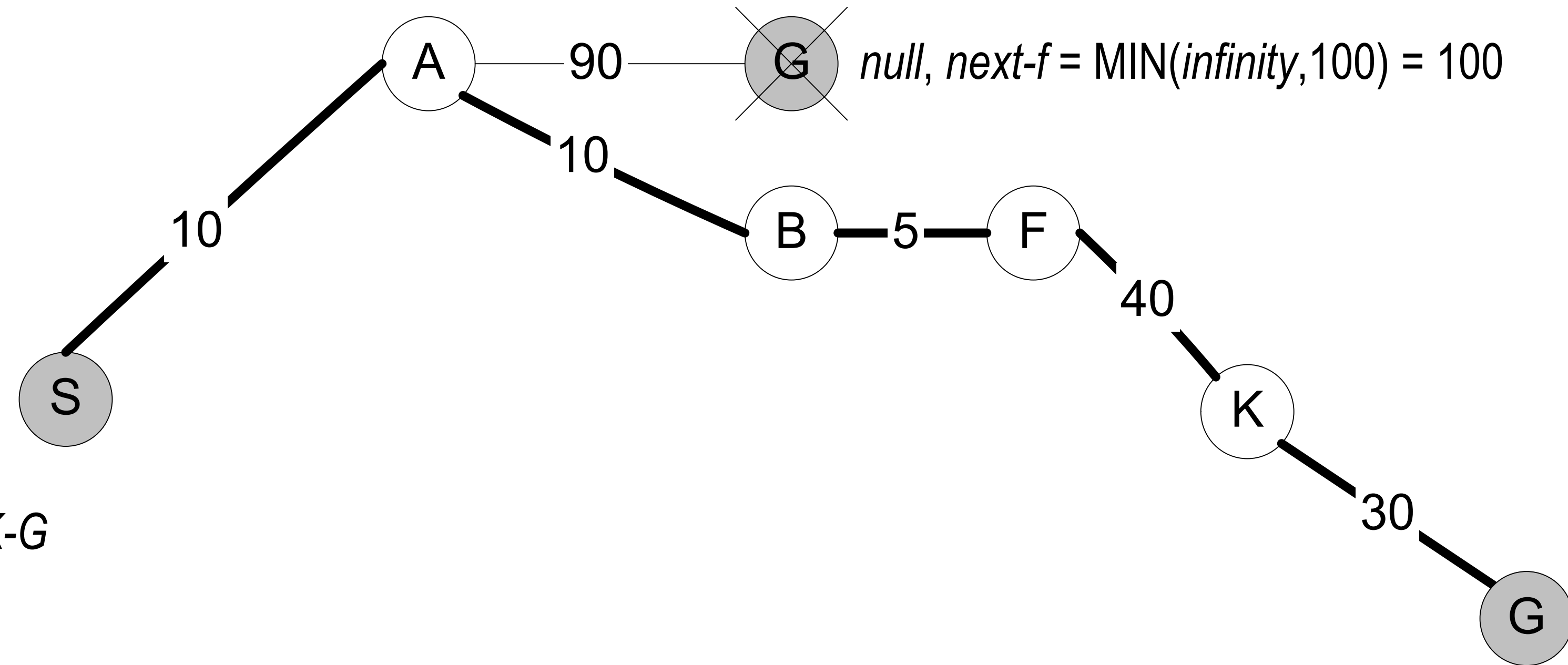
Langkah 5:

inputs:

- *node* = S
- *f-limit* = 95

returns:

- *solusi*: S-A-B-F-K-G
- *next-f* = 95



Total Biaya = 95

Biaya optimal = 95

CAK2HAB3 - Dasar Kecerdasan Artifisial

Bidirectional A*

S1 Informatika – Fakultas Informatika

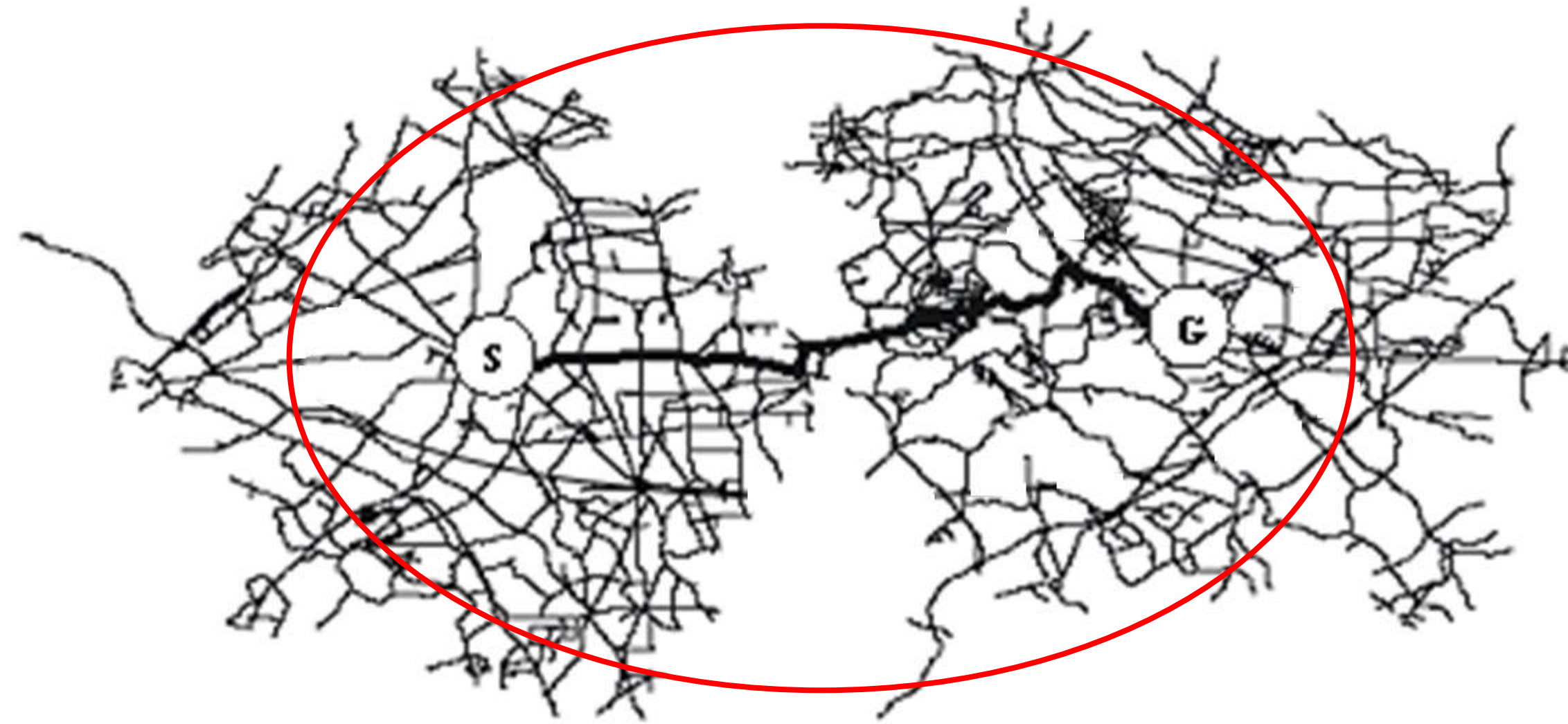
Semester Genap 2024/2025



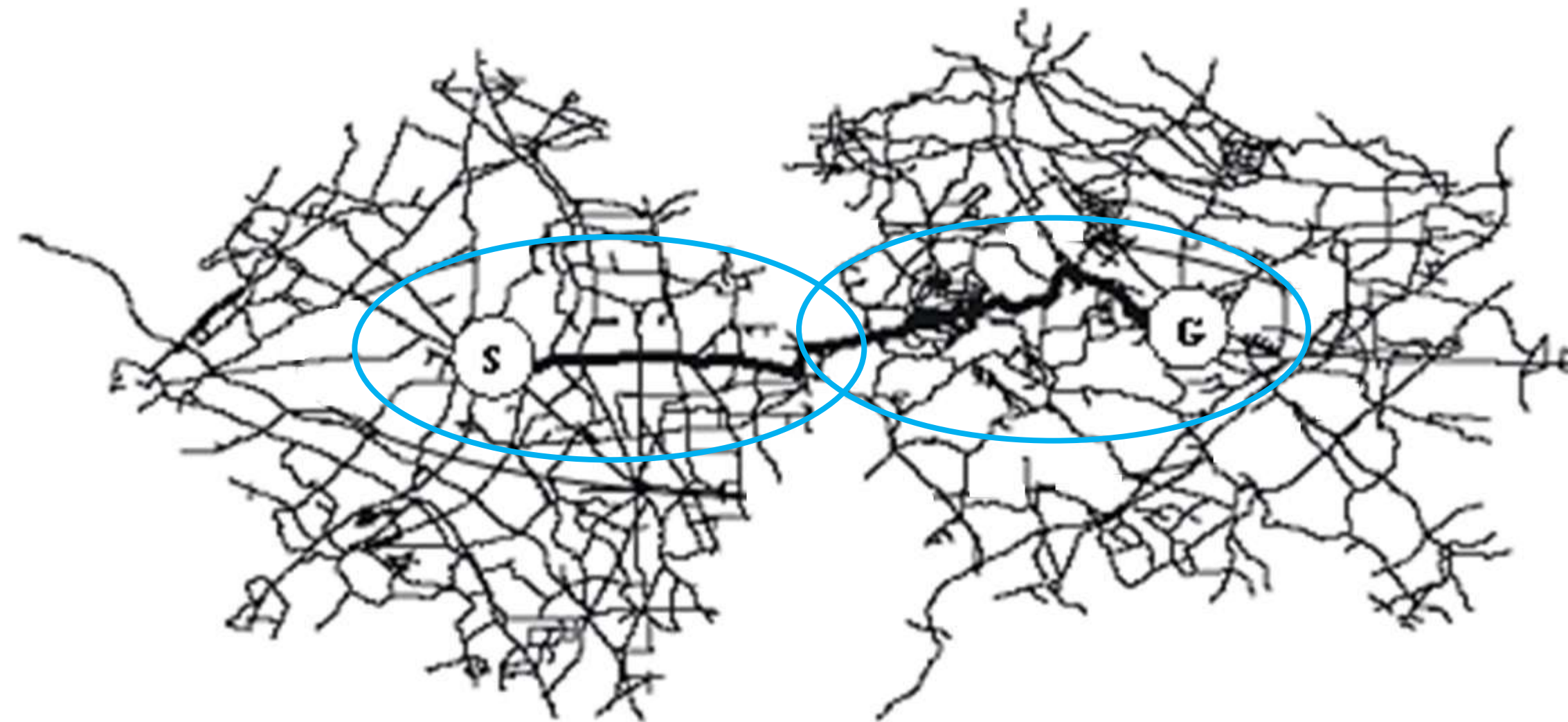


Bidirectional A* (BDA*)

- ▶ BDA* melakukan pencarian dari dua arah
 - Pencarian maju: dari *start* ke *goal*
 - Pencarian mundur: dari *goal* ke *start*
 - Pencarian dihentikan jika *BestNode* dari *start* telah berada di dalam *CLOSED* dari *goal*. Cek apakah harus mengganti *parent*-nya *BestNode* tersebut dari arah *goal*
 - Atau sebaliknya, pencarian dihentikan jika *BestNode* dari *goal* telah berada di dalam *CLOSED* dari *start*. Cek apakah harus mengganti *parent*-nya *BestNode* tersebut dari arah *start*
- 



(a) Area pencarian A*



(b) Area pencarian BDA*

function BDA*(*masalah*) **returns** *solusi*

inputs: *masalah*: berisi ruang masalah, jenis solusi, S, dan G

local variables: *BNs* dan *BNg*: *BestNode* dari arah start dan goal

OPENs dan *OPENg*: *OPEN* dari arah start dan goal

CLOSEDs dan *CLOSEDg*: *CLOSED* dari arah start dan goal

loop sampai goal ditemukan

BNs, *OPENs*, *CLOSEDs* $\leftarrow$ A*(*OPENs*,*CLOSEDs*,S,G)

if *BNs* **in** *CLOSEDg* **then** { goal sudah ditemukan }

v $\leftarrow$ simpul di *CLOSEDg* yang sama dengan *BNs*

g-terbaik $\leftarrow$ *g*(G,*v*) { biaya sebenarnya dari G ke *v* melalui *parent* lama }

loop untuk semua *suk* = simpul di *CLOSEDg* yang merupakan suksesor *v*

if *g*(G,*v*) melalui *suk* < *g*-terbaik **then**

g-terbaik $\leftarrow$ *g*(G,*v*) melalui *suk*

parent-baru $\leftarrow$ *suk*

end

end

g(*v*) $\leftarrow$ *g*-terbaik

parent dari *v* $\leftarrow$ *parent*-baru

returns solusi

end

BNg, *OPENg*, *CLOSEDg* $\leftarrow$ A*(*OPENg*,*CLOSEDg*,G,S)

if *BNg* **in** *CLOSEDs* **then** { goal sudah ditemukan }

u $\leftarrow$ simpul di *CLOSEDs* yang sama dengan *BNg*

g-terbaik $\leftarrow$ *g*(S,*u*) { biaya sebenarnya dari S ke *u* melalui *parent* lama }

loop untuk semua *suk* = simpul di *CLOSEDg* yang merupakan suksesor *u*

if *g*(S,*u*) melalui *suk* < *g*-terbaik **then**

g-terbaik $\leftarrow$ *g*(S,*u*) melalui *suk*

parent-baru $\leftarrow$ *suk*

end

end

g(S,*u*) $\leftarrow$ *g*-terbaik

parent dari *u* $\leftarrow$ *parent*-baru

returns solusi

end

end

function BDA*(*masalah*) **returns** *solusi*

inputs: *masalah*: berisi ruang masalah, jenis solusi, S, dan G

local variables: *BNs* dan *BNg*: *BestNode* dari arah start dan goal

OPENs dan *OPENg*: *OPEN* dari arah start dan goal

CLOSEDs dan *CLOSEDg*: *CLOSED* dari arah start dan goal

loop sampai goal ditemukan

BNs, *OPENs*, *CLOSEDs* $\leftarrow$ A*(*OPENs*,*CLOSEDs*,S,G)

if *BNs* **in** *CLOSEDg* **then** { goal sudah ditemukan }

v $\leftarrow$ simpul di *CLOSEDg* yang sama dengan *BNs*

g-terbaik $\leftarrow$ *g*(G,*v*) { biaya sebenarnya dari G ke *v* melalui *parent* lama }

loop untuk semua *suk* = simpul di *CLOSEDg* yang merupakan suksesor *v*

if *g*(G,*v*) melalui *suk* < *g*-terbaik **then**

g-terbaik $\leftarrow$ *g*(G,*v*) melalui *suk*

parent-baru $\leftarrow$ *suk*

end

end

g(*v*) $\leftarrow$ *g*-terbaik

parent dari *v* $\leftarrow$ *parent*-baru

returns solusi

end

BNg, *OPENg*, *CLOSEDg* $\leftarrow$ A*(*OPENg*,*CLOSEDg*,G,S)

if *BNg* **in** *CLOSEDs* **then** { goal sudah ditemukan }

u $\leftarrow$ simpul di *CLOSEDs* yang sama dengan *BNg*

g-terbaik $\leftarrow$ *g*(S,*u*) { biaya sebenarnya dari S ke *u* melalui *parent* lama }

loop untuk semua *suk* = simpul di *CLOSEDg* yang merupakan suksesor *u*

if *g*(S,*u*) melalui *suk* < *g*-terbaik **then**

g-terbaik $\leftarrow$ *g*(S,*u*) melalui *suk*

parent-baru $\leftarrow$ *suk*

end

end

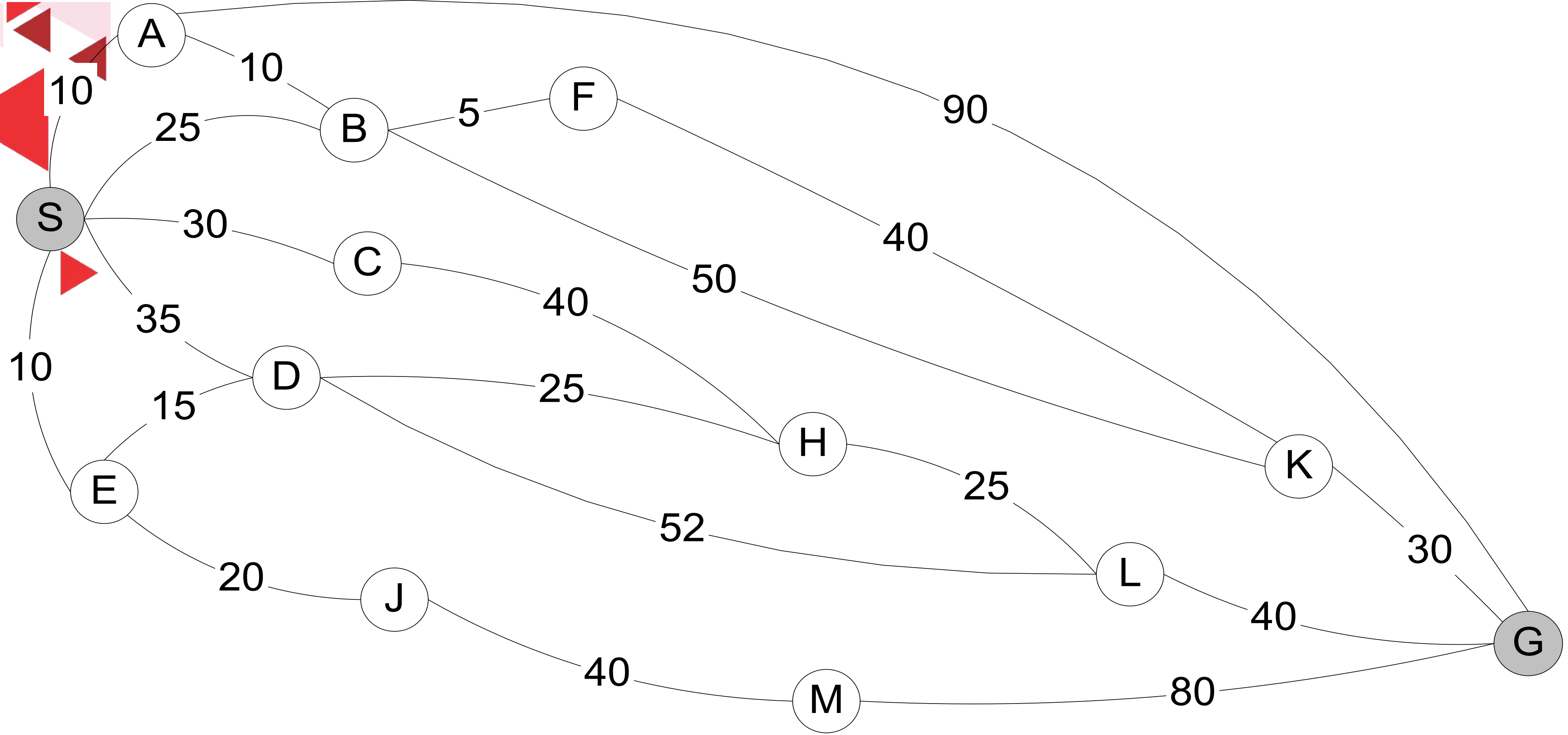
g(S,*u*) $\leftarrow$ *g*-terbaik

parent dari *u* $\leftarrow$ *parent*-baru

returns solusi

end

end

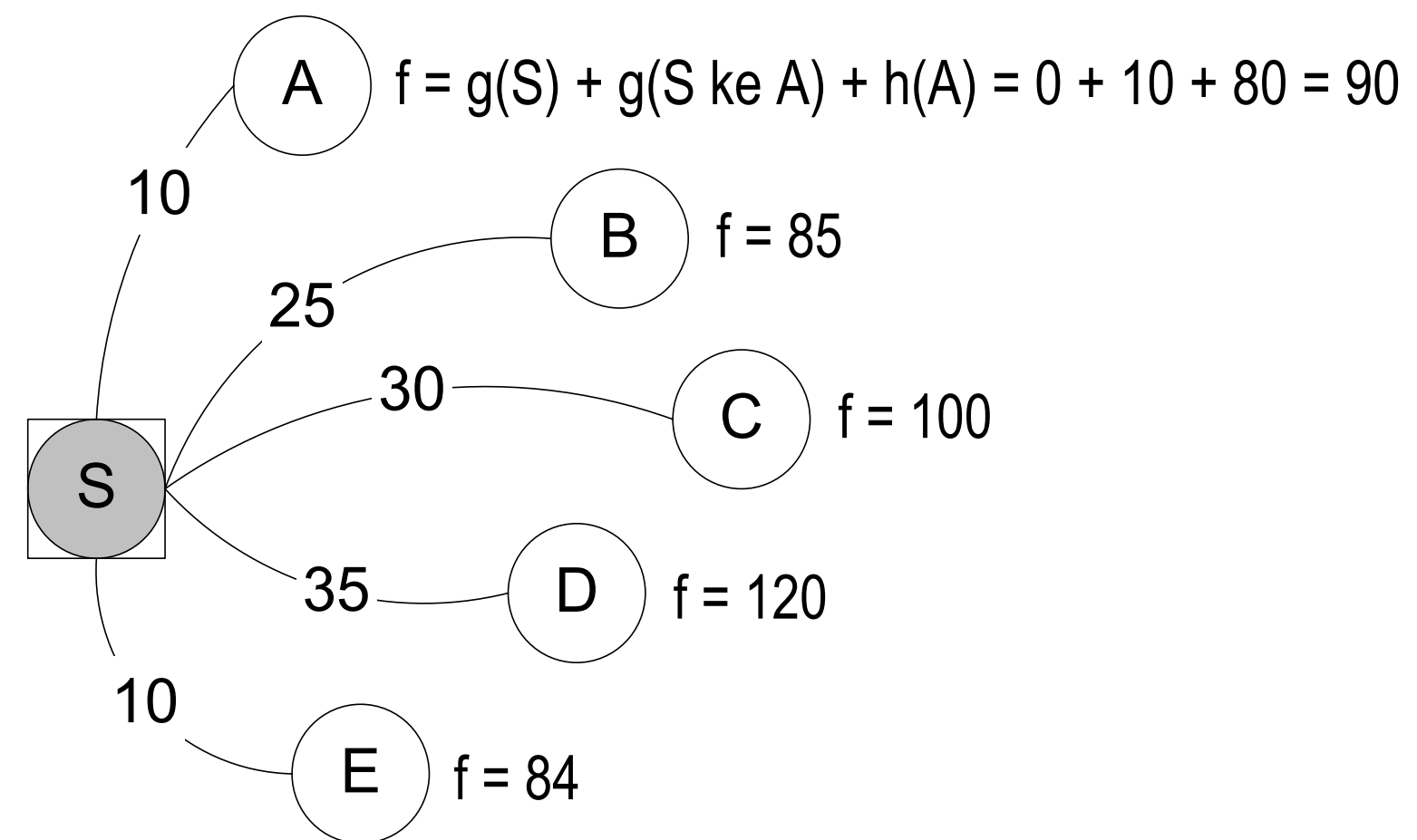


n	S	A	B	C	D	E	F	G	H	J	K	L	M
$h_s(n)$	80	80	60	70	85	74	70	0	40	100	30	20	70

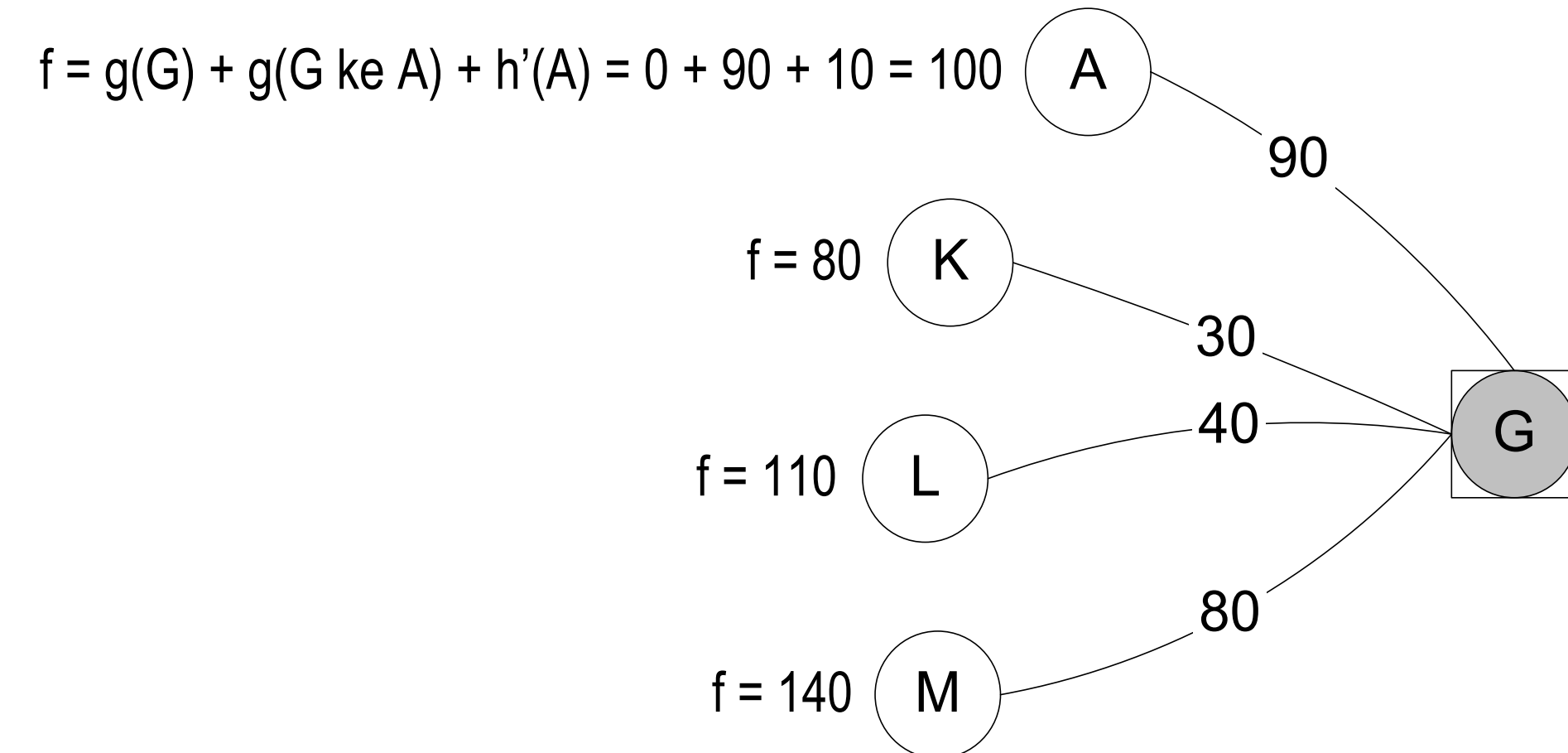
n	S	A	B	C	D	E	F	G	H	J	K	L	M
$h_g(n)$	0	10	15	25	30	5	20	90	45	25	50	70	60

Pencarian Maju (dari S ke G)

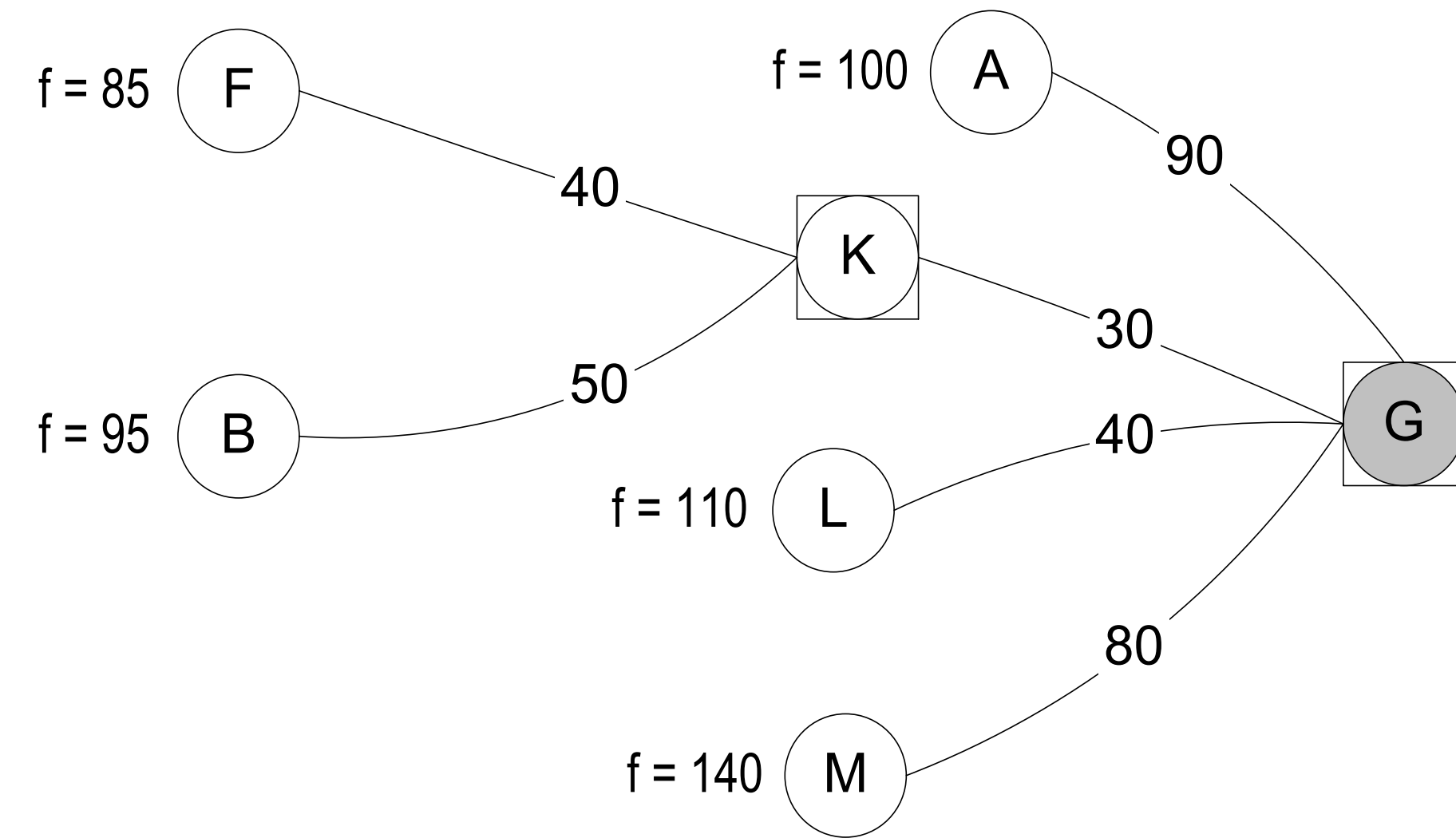
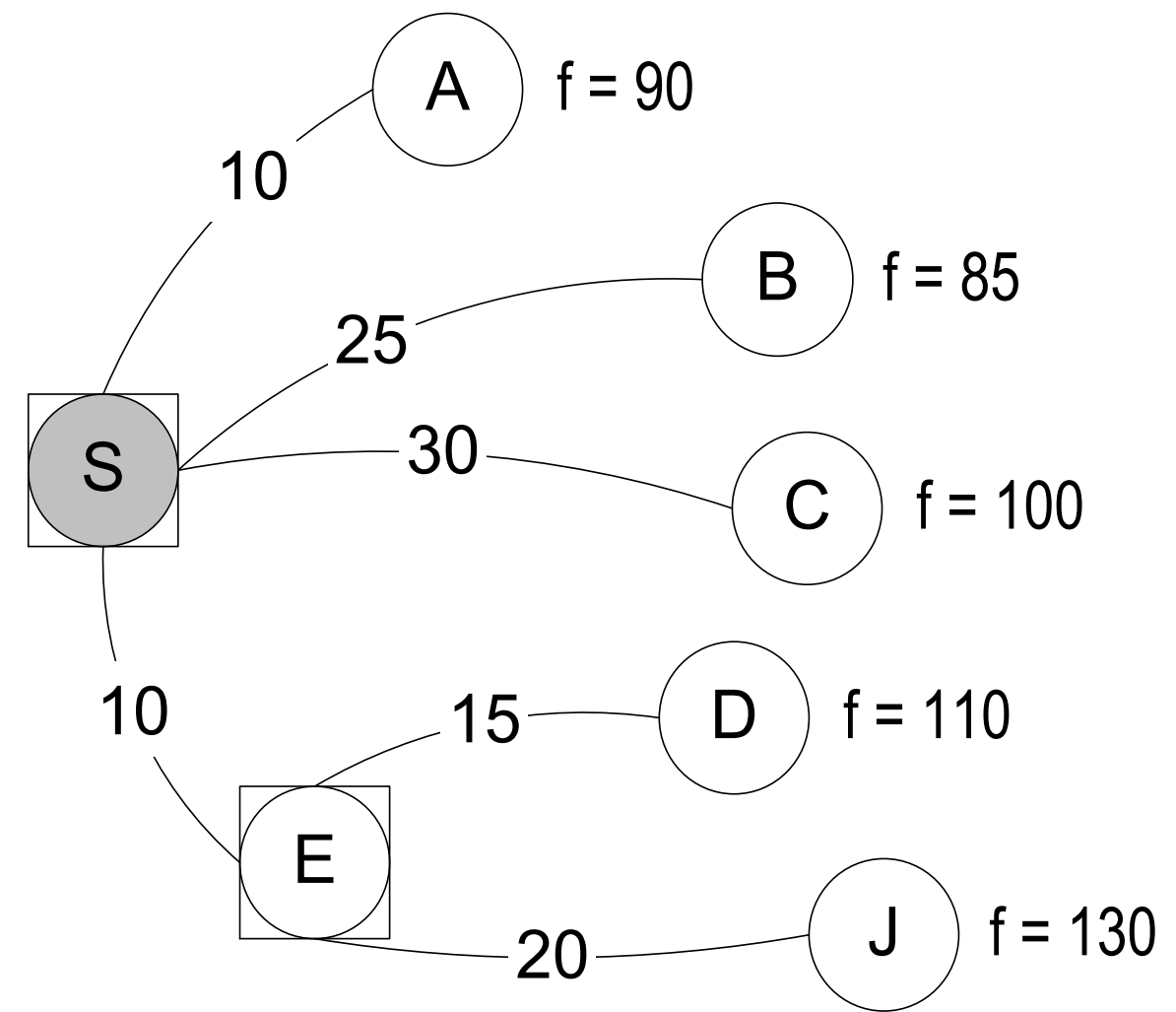
Langkah 1



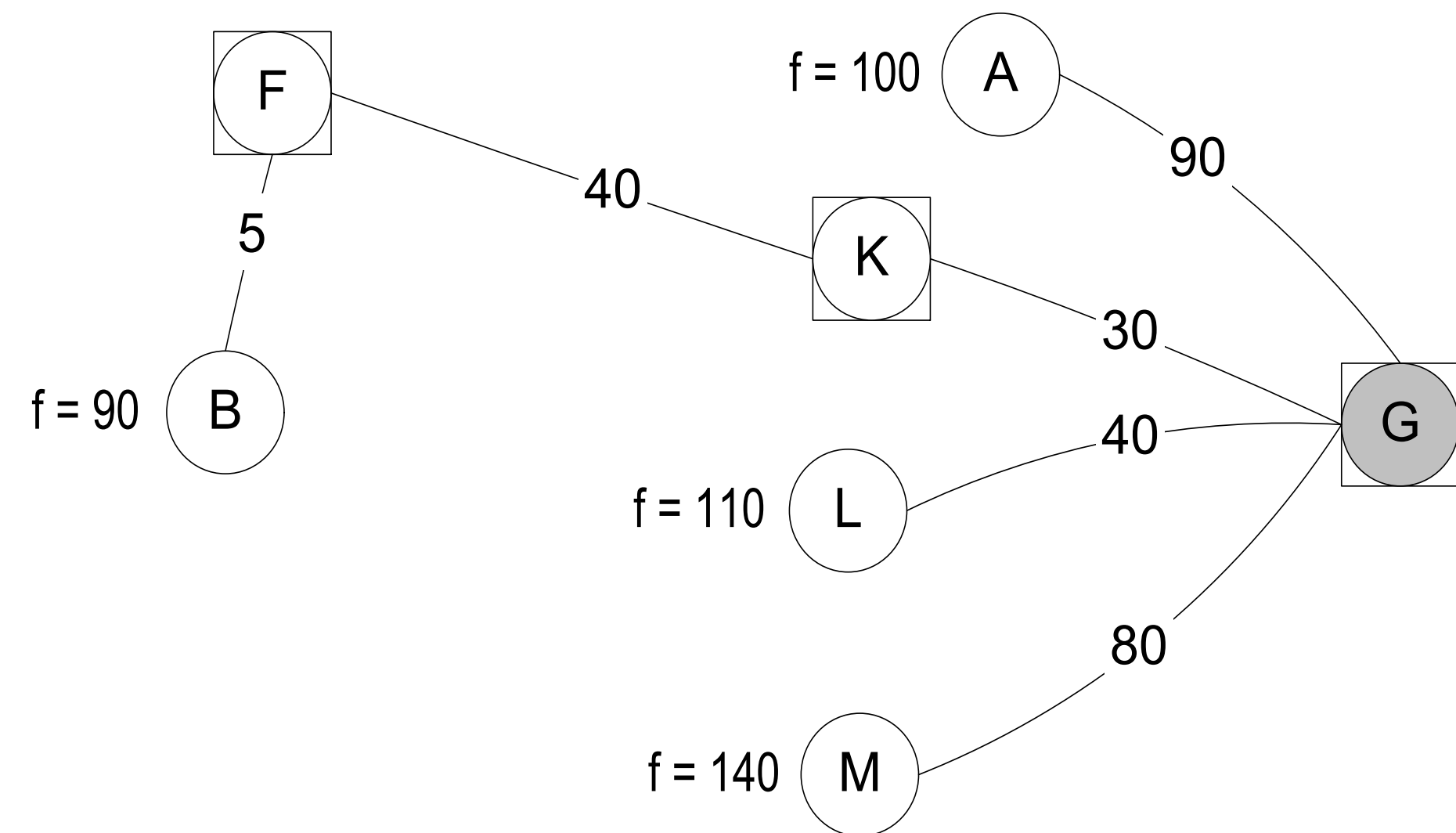
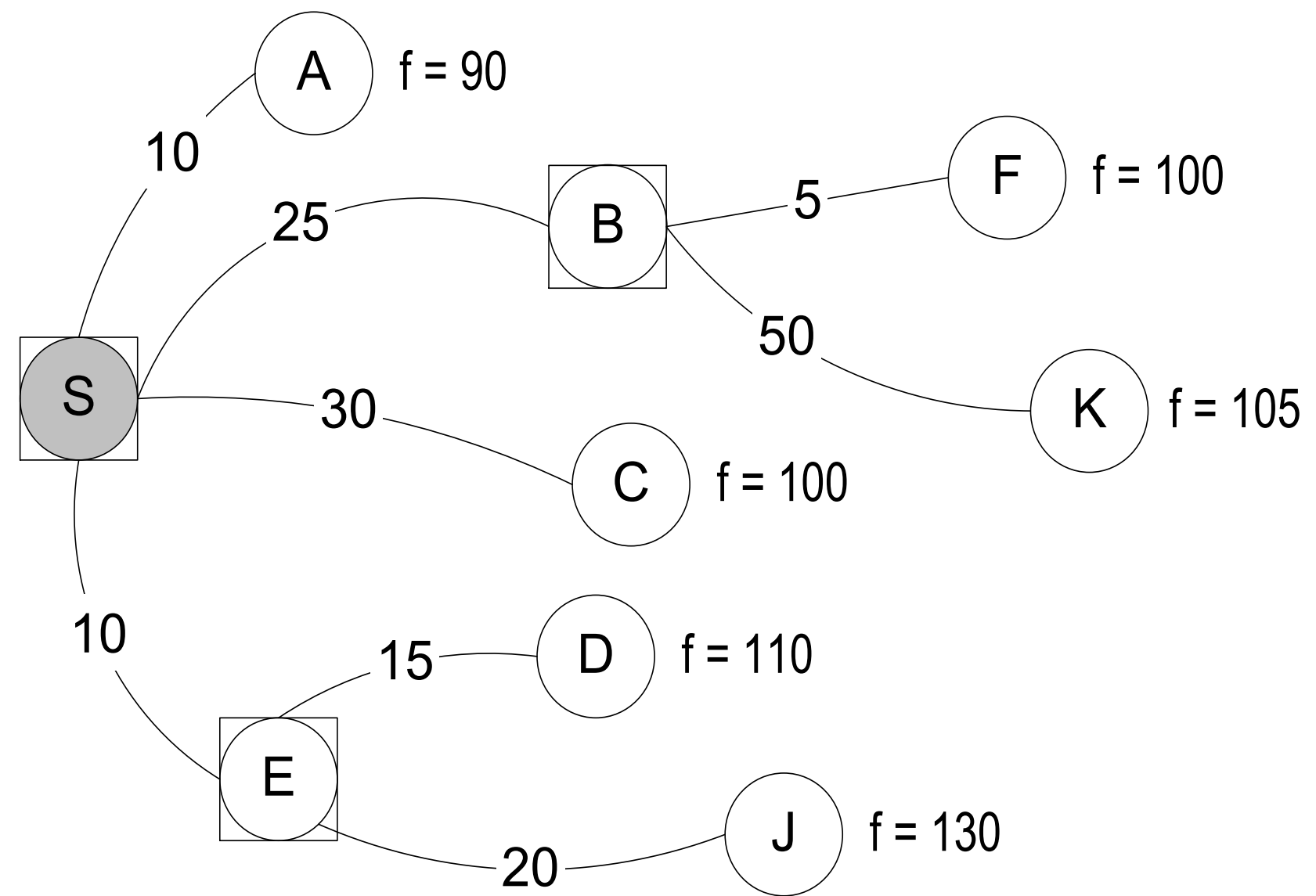
Pencarian Mundur (dari G ke S)



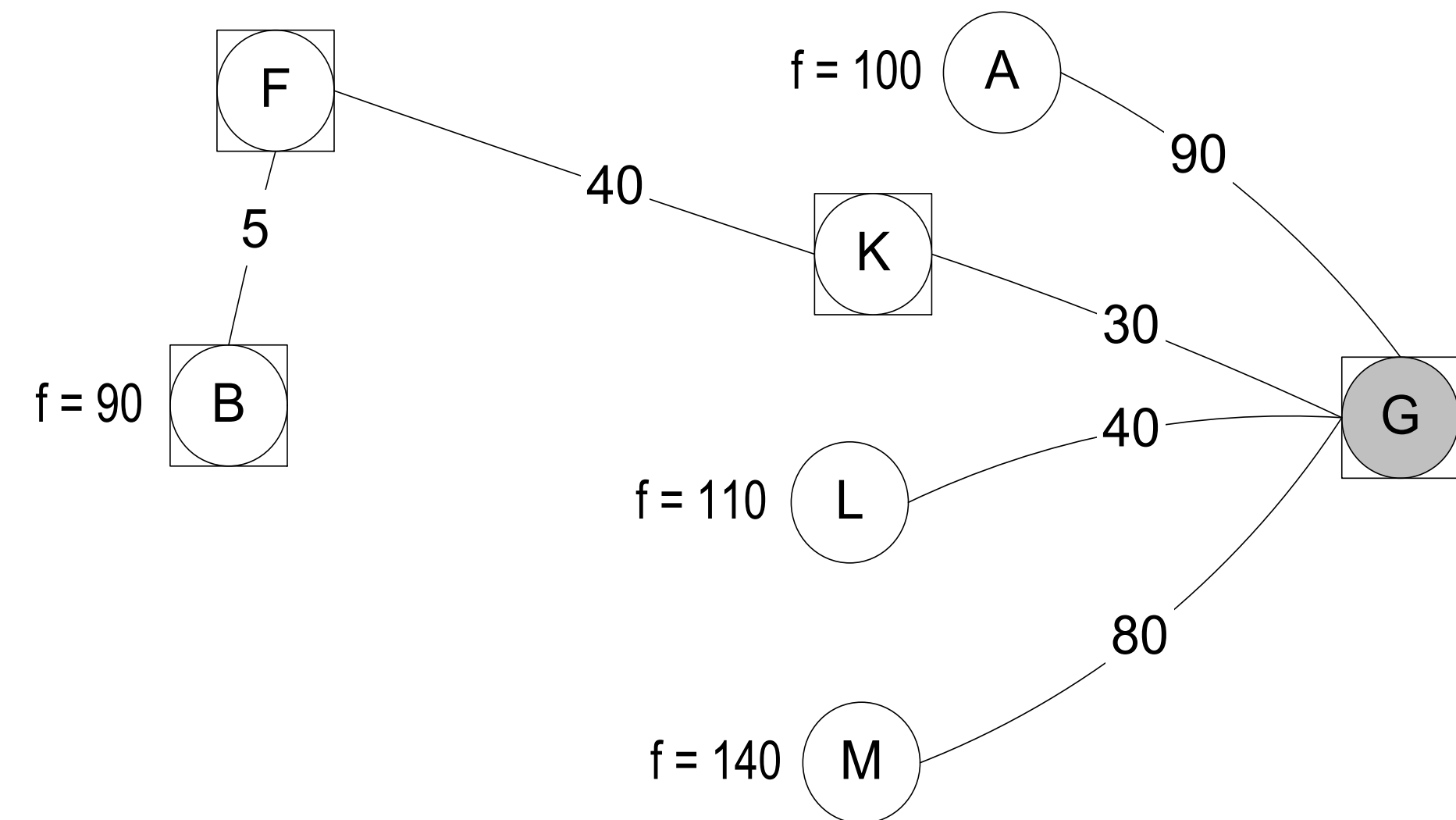
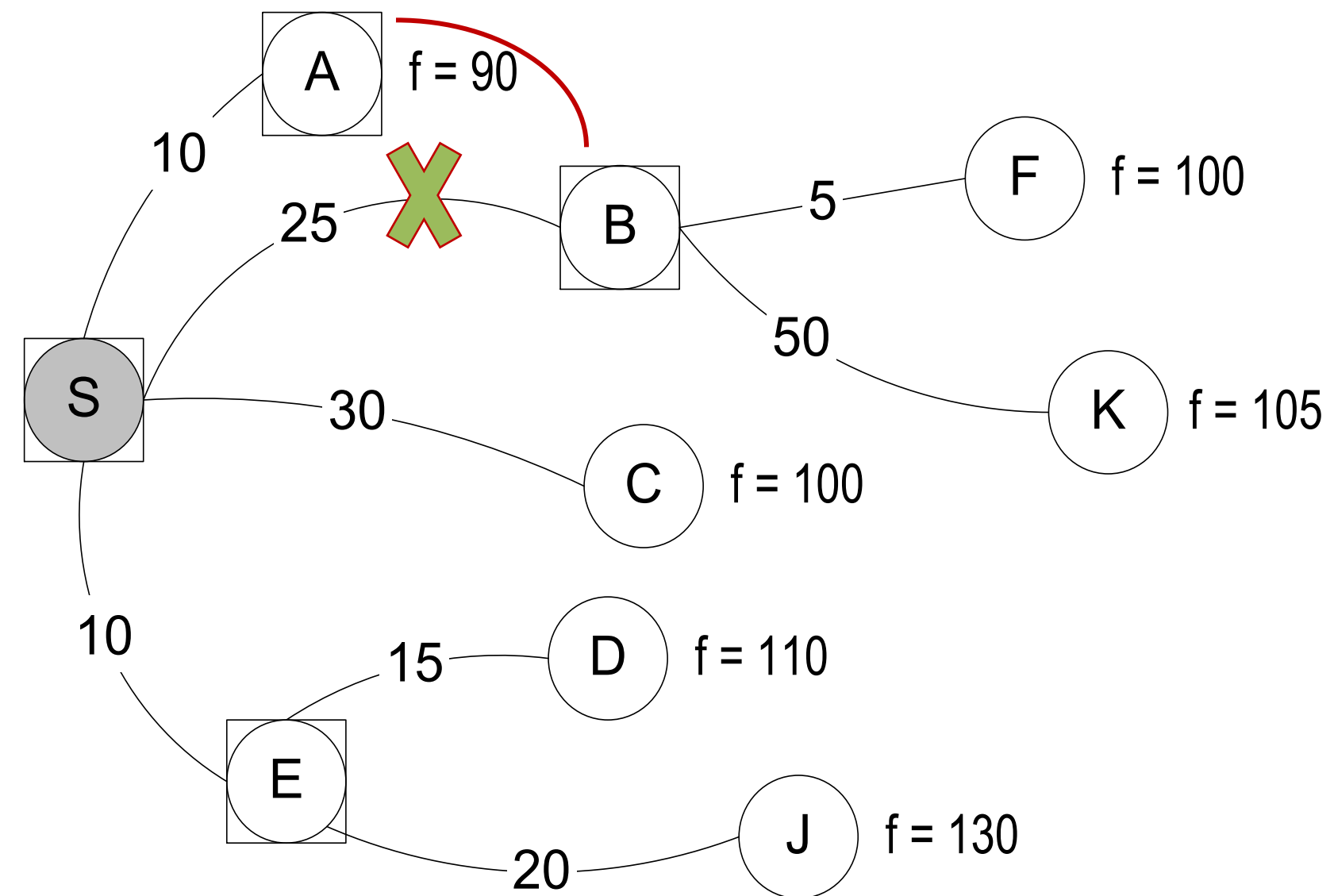
Langkah 2



Langkah 3



Langkah 4



Rute: S-A-B-F-K-G dengan total biaya = 95

Biaya optimal = 95



Modified Bi-directional A* (MBDA*)

- Fungsi heuristik untuk simpul n pada pencarian maju (dari S ke G):

$$f = g(S, n) + \frac{1}{2} [h_s(n) - h_g(n)]$$

- Fungsi heuristik untuk simpul n pada pencarian mundur (dari G ke S):

$$f = g(G, n) + \frac{1}{2} [h_g(n) - h_s(n)]$$




Terima Kasih

